

Reward-tilted steering for diffusion generative models and Branching Flows

Lukas Billera

June 16, 2026

ABSTRACT

We give an introduction to stochastic processes and their use in diffusion generative models. We discuss stochastic interpolants, infinitesimal generators, and Latent Process Generator Matching, where an auxiliary process is used during training and then marginalized out. Standard diffusion models require ad hoc choices when the length of a sequence is not known *a priori*, for example the length of a natural language response or the number of atoms in a molecule. For these cases, we present Branching Flows, where elements evolve, branch, and die with rates learned by the model. During inference this lets the model determine the number of elements. Finally, we discuss Feynman–Kac steering for sampling from reward-tilted distributions and its use with Branching Flows. We empirically test simplifications of the theory in this setting.

1 INTRODUCTION

Generative models start from a philosophical premise. Although the “set” of all images of cats or possible three-dimensional protein structures has no clean mathematical description, one may postulate the existence of an underlying probability distribution p_{data} on it that assigns higher probability to natural/realistic structures or images than to unnatural or unrealistic ones. A dataset $\mathcal{D} = \{x^{(1)}, \dots, x^{(N)}\}$, such as the Protein Data Bank Berman et al. (2000), is then considered to be an i.i.d. sample from p_{data} .

The *generative-modeling problem* is to construct, from $\mathcal{D}$, a procedure that produces fresh samples $X \sim p_{\text{data}}$. The same setup applies to images, molecules, or natural language, with p_{data} the hypothetical distribution one wishes to sample from. A diffusion generative model is an attempt to solve the generative modelling problem by learning a *diffusion process* (which is a stochastic process akin to a small-step limit of a random walk) that transports pure noise (so e.g., samples from a Gaussian) to samples from p_{data} . We discuss some of the basic theory underlying stochastic processes and diffusion processes in §2.

For historical orientation, in early formulations of diffusion Song et al. (2021); Karras et al. (2022); Ho et al. (2020), this was accomplished via attempting to reverse a *noising/corrupting process*, which turns samples from p_{data} into ones that are approximately Gaussian, i.e. noise. The idea is that the *reverse* of a process which turns samples from p_{data} into ones that are approximately Gaussian is in turn a process that (approximately) turns samples from a Gaussian into samples from p_{data} . This was generally accomplished through employing a diffusion time-reversal result (Anderson, 1982) to obtain a neural-network training objective, and while the underlying idea of reversing the noising process to obtain the generative process is intuitive, it introduces complications for the purposes of exposition and reduces the generality of these models. These approaches include many classes of “score-matching” or “score-based” generative models, since the key training target becomes a quantity known as the score $\nabla \log p_t(x)$, or gradient of the logarithm of the probability density, subject to various levels of noise, e.g. $p_t := p_{\text{data}} * \mathcal{N}(0, \sigma_t^2)$, assumed to be a continuous density.

However, there is no need for the complication of a time-reversal or the score, as pioneered through the “stochastic interpolants” lens of diffusion (Albergo et al., 2023). In this view of diffusion, the generative modelling task is to learn a diffusion process that transports samples from a simple distribution p_{simple} (e.g., a Gaussian) to the data distribution p_{data} as time ranges from $t = 0$ to $t = 1$. But in this formalism, this is not thought of as reversing a process which turns samples from p_{data} into samples from p_{simple} . Instead, one specifies tractable *stochastic interpolant* paths which are diffusion processes that start from a sample of p_{simple} , at time $t = 0$, and terminate at a sample from p_{data} , at time $t = 1$. One uses these interpolant paths to construct a neural-network training objective which, at the loss minimum of an idealized population objective and under the assumptions discussed below, identifies a marginal process

with the desired endpoints. A trained neural network is then an approximation to this ideal process. We discuss the stochastic interpolants view of diffusion in further detail in §3.1.

Throughout the thesis, the *idealized population setting* means that the exact objective with the hypothetical p_{data} is minimized over a sufficiently rich model class, all stated integrability and regularity assumptions hold for the resulting marginal generator, and the relevant Kolmogorov forward equation has the required uniqueness property. Actual neural-network training, finite data, and time discretization are additional approximations to this setting. In the simplest case, a diffusion process with no noise is still allowed in this formalism, in which case the interpolant path is simply a curve connecting a sample of p_{simple} at time $t = 0$ and a sample of p_{data} at time $t = 1$. The simple case when there is no noise in the stochastic interpolant path is known as Flow Matching (Lipman et al., 2023), in which case the stochastic interpolant is simply a curve (typically a line) which connects a sample from p_{simple} and a sample from p_{data} .

Next, the stochastic interpolants viewpoint can be expanded to a broader class of interpolating processes than only diffusion processes on $\mathbb{R}^n$, stated at the level of infinitesimal generators. An infinitesimal generator describes the infinitesimal evolution of a stochastic process viewed through the lens of test functions, and thereby reduces the “complexity” of the state space. Through this formalism, known as Generator Matching (Holderrieth et al., 2025), one can cover diffusions and jump processes on a wide variety of state spaces. Here we use a formulation of this known as Latent Process Generator Matching (Billera et al., 2026a), in which auxiliary information can evolve stochastically alongside the interpolant path.

After this, §4 explains what it means to “train” a neural network. Training is the optimization of a differentiable loss function through techniques such as stochastic gradient descent (Robbins and Monro, 1951). This connects the theory with the practical problem of turning a principled training objective, meaning a loss function whose minimizer in the idealized population setting has a desired property, into a neural network that approximately has the desired property because it approximately minimized that loss. We do not, however, attempt to characterize the “closeness” of such approximations, which is instead largely driven by experiments.

Using this latent-process formulation, we describe Branching Flows (Billera et al., 2026c) as a way to transport samples from p_{simple} to p_{data} when the state space has variable dimension, for instance $\{(n, x) : x \in (\mathbb{R}^d)^n, 1 \leq n \leq N\}$. This is natural for tasks such as molecular generation, where the number of atoms in a molecule targeted at a given design task is not known in advance, and should ideally be determined by the model. In the branching context, each *element* $x_i \in \mathbb{R}^d$ in the sequence $(n, x_1, \dots, x_n)$ evolves stochastically between admissible split events, for $n < N$ and $1 \leq j \leq n$,

$$\text{split}_j(n, x_1, \dots, x_n) = (n + 1, x_1, \dots, x_j, x_j, x_{j+1}, \dots, x_n)$$

and admissible deletion events, for $n > 1$ and $1 \leq j \leq n$,

$$\text{del}_j(n, x_1, \dots, x_n) = (n - 1, x_1, \dots, x_{j-1}, x_{j+1}, \dots, x_n)$$

that occur with rates parametrized by the model/neural network.

The learned rates come from interpolant paths in which elements evolve, split, and delete so that a path starting at a sample $(n, x_1, \dots, x_n)$ from p_{simple} terminates at a sample $(m, y_1, \dots, y_m)$ from p_{data} . We describe these paths, their latent variables, the training objective, and the sampling algorithms.

Finally, once equipped with the ability to approximately generate samples from the data distribution p_{data} , one can ask if it is possible to do better. Given a reward function $r(x)$, one may for example wish to sample from the reward-tilted distribution $\frac{1}{Z_{\text{FK}}} e^{\beta r(x)} p_{\text{data}}$, where in this context a higher reward corresponds to desirable attributes, such as politeness for a language model or binding affinity for a protein binder. One technique for doing so, without the necessity for retraining the model, is known as Feynman–Kac steering (Singhal et al., 2025). We discuss the applicability of Feynman–Kac steering to Branching Flows, and give qualitative results indicating a practical version of this technique employed in Proteina Complexa (Didi et al., 2026), a state-of-the-art algorithm for protein design, remains effective when applied to Branching Flows.

2 MATHEMATICAL BACKGROUND ON STOCHASTIC PROCESSES

2.1 BASIC BACKGROUND

We begin with a brief primer on the measure-theoretic foundations of probability, which underlie stochastic processes and, in turn, diffusion generative models.

Definition 2.1.1 (Definition 2.1.1. Øksendal (1998)). If Ω is a set, then a σ -algebra on Ω is a collection $\mathcal{F}$ of subsets of Ω such that

- i) $\emptyset \in \mathcal{F}$
- ii) $F \in \mathcal{F}$ implies $F^c = \Omega \setminus F \in \mathcal{F}$.
- iii) $A_1, A_2, \dots \in \mathcal{F}$ implies $\bigcup_{i=1}^{\infty} A_i \in \mathcal{F}$.

A *measurable space* is a pair $(\Omega, \mathcal{F})$. A probability measure $\mathbb{P}$ on a measurable space is a function $\mathbb{P} : \mathcal{F} \rightarrow [0, 1]$ such that

- i) $\mathbb{P}(\emptyset) = 0, \mathbb{P}(\Omega) = 1$.
- ii) If $A_1, A_2, \dots \in \mathcal{F}$ are disjoint, then

$$\mathbb{P}\left(\bigcup_{i=1}^{\infty} A_i\right) = \sum_{i=1}^{\infty} \mathbb{P}(A_i).$$

A triple $(\Omega, \mathcal{F}, \mathbb{P})$ is called a *probability space*.

Definition 2.1.2. The smallest σ -algebra on Ω containing a collection $\mathcal{C}$ of subsets of Ω is the smallest σ -algebra containing $\mathcal{C}$. Mathematically, it is the intersection of all σ -algebras containing $\mathcal{C}$, which can be shown to again be a σ -algebra. This is denoted $\sigma(\mathcal{C})$ and is called the σ -algebra *generated* by $\mathcal{C}$.

Definition 2.1.3. The Borel σ -algebra $\mathcal{B}(S)$ of a topological space $(S, \mathcal{T})$ is the smallest σ -algebra containing the topology $\mathcal{T}$, i.e. the σ -algebra generated by the collection $\mathcal{T}$, $\mathcal{B}(S) = \sigma(\mathcal{T})$.

Definition 2.1.4. A function $f : \Omega \rightarrow S$, where $(\Omega, \mathcal{F})$ and $(S, \mathcal{E})$ are measurable spaces is called measurable if $f^{-1}(E) \in \mathcal{F}$ for every $E \in \mathcal{E}$. When $\mathcal{E} = \mathcal{B}(S)$, then f is called Borel-measurable.

Remark 2.1.5. We often write $f : (\Omega, \mathcal{F}) \rightarrow (S, \mathcal{E})$ to make explicit the σ -algebras with respect to which f is measurable.

Definition 2.1.6. Let $(S_1, \mathcal{E})$ and $(S_2, \mathcal{F})$ be measurable spaces. The product σ -algebra $\mathcal{E} \otimes \mathcal{F}$ is the σ -algebra generated by the collection of rectangles $E \times F$ with $E \in \mathcal{E}$ and $F \in \mathcal{F}$, i.e.,

$$\mathcal{E} \otimes \mathcal{F} = \sigma(\{E \times F : E \in \mathcal{E}, F \in \mathcal{F}\}).$$

Definition 2.1.7 ((Bogachev, 2007, Lemma 2.12.2)). The σ -algebra on a set Ω generated by a collection of functions $\mathcal{F}$ from Ω to a measurable space $(S, \mathcal{E})$ is denoted by $\sigma(\mathcal{F})$ and is the intersection of all σ -algebras $\mathcal{G}$ on Ω such that every $f \in \mathcal{F}$ is measurable as a map $(\Omega, \mathcal{G}) \rightarrow (S, \mathcal{E})$.

The Doob-Dynkin lemma (below) characterizes exactly which functions are $\sigma(\mathcal{F})$ -measurable. Namely, it is the Borel-measurable functions only depending on the values of f , for $f \in \mathcal{F}$. We state a form of (Bogachev, 2007, Lemma 2.12.3), here using a finite collection of functions, for simplicity.

Lemma 2.1.8 (Doob-Dynkin). *Let $\mathcal{F} = \{f_1, \dots, f_n\}$ be a finite collection of real-valued functions $f_i : \Omega \rightarrow \mathbb{R}$. Then a real-valued function $g : \Omega \rightarrow \mathbb{R}$ is $\sigma(\mathcal{F})$ -measurable if and only if there exists a Borel-measurable $\psi : \mathbb{R}^n \rightarrow \mathbb{R}$ such that*

$$g(\omega) = \psi(f_1(\omega), \dots, f_n(\omega)).$$

Definition 2.1.9 ((Bogachev, 2007, Definition 6.2.10(i))). An isomorphism of two measurable spaces $(X, \mathcal{A})$ and $(Y, \mathcal{B})$ is a bijection $j : X \rightarrow Y$ such that j is measurable and j^{-1} is measurable.

Definition 2.1.10. A Polish space S is a topological space that is separable (i.e., has a countable dense set) and completely metrizable (i.e., it holds that the topology of S can be induced by a metric d in which (S, d) is a complete metric space).

For example, $\mathbb{R}^n$ is a Polish space. The Doob-Dynkin lemma extends to functions valued in Polish spaces, through the following:

Lemma 2.1.11 ((Bogachev, 2007, Corollary 6.8.8)). *Any two uncountable Polish spaces equipped with their Borel σ -algebras are isomorphic, in the sense of Definition 2.1.9.*

Corollary 2.1.12 (Doob-Dynkin, Polish-valued). *Let $\mathcal{F} = \{f_1, \dots, f_n\}$ be real-valued functions $f_i : \Omega \rightarrow \mathbb{R}$ and let S be an uncountable Polish space. Then $g : \Omega \rightarrow S$ is $\sigma(\mathcal{F})$ -measurable if and only if there exists a Borel-measurable $\psi : \mathbb{R}^n \rightarrow S$ such that $g = \psi(f_1, \dots, f_n)$.*

Proof. The if direction is immediate, since the composition of measurable maps is measurable. For only if, since S is an uncountable Polish space, Lemma 2.1.11 gives a measure-space isomorphism $j : S \rightarrow \mathbb{R}$ with Borel-measurable inverse $j^{-1} : \mathbb{R} \rightarrow S$. We have that $j \circ g : \Omega \rightarrow \mathbb{R}$ is $\sigma(\mathcal{F})$ -measurable, so the real-valued Doob-Dynkin lemma (Lemma 2.1.8) yields a Borel $\tilde{\psi} : \mathbb{R}^n \rightarrow \mathbb{R}$ with $j \circ g = \tilde{\psi}(f_1, \dots, f_n)$. Setting $\psi := j^{-1} \circ \tilde{\psi}$, which is Borel-measurable, gives $g = j^{-1} \circ (j \circ g) = \psi(f_1, \dots, f_n)$. $\square$

Definition 2.1.13. Let S be a Polish space and $(\Omega, \mathcal{F}, \mathbb{P})$ a probability space. An S -valued random variable is a Borel-measurable function $X : (\Omega, \mathcal{F}) \rightarrow (S, \mathcal{B}(S))$.

Definition 2.1.14. Given a measure μ on $(\Omega, \mathcal{F})$ and a measurable map $f : (\Omega, \mathcal{F}) \rightarrow (S, \mathcal{E})$, the *pushforward* (or image) measure of μ under f is $f_{\#}\mu := \mu \circ f^{-1}$, so that $f_{\#}\mu(E) = \mu(f^{-1}(E))$ for $E \in \mathcal{E}$.

Definition 2.1.15. The law of a random variable $X : (\Omega, \mathcal{F}) \rightarrow (S, \mathcal{B}(S))$ on the probability space $(\Omega, \mathcal{F}, \mathbb{P})$ is the pushforward measure $X_{\#}\mathbb{P} = \mathbb{P} \circ X^{-1}$. We write $X \sim \mu$ when $\mu = \text{Law}(X)$. If two random variables X and Y have the same law then they are said to be equal in law (or, distribution), which is written $X \stackrel{d}{=} Y$.

Now we shift back to $S = \mathbb{R}^d$, and define the basic quantities which quantify random variables probabilistically.

Definition 2.1.16. Let $(\Omega, \mathcal{F}, \mathbb{P})$ be a probability space. The expected value of a random variable $X : (\Omega, \mathcal{F}) \rightarrow (\mathbb{R}^d, \mathcal{B}(\mathbb{R}^d))$ is

$$\mathbb{E}[X] = \int_{\Omega} X \, d\mathbb{P}.$$

Remark 2.1.17. In the above, the integral is the Lebesgue integral of X against the probability measure $\mathbb{P}$. We refer the reader to any text on measure theory, e.g. Rudin (1987) if they are not familiar with Lebesgue integration.

Definition 2.1.18 (Conditional expectation (Bogachev, 2007, Def. 10.1.1)). Let $(\Omega, \mathcal{F}, \mathbb{P})$ be a probability space, let X be an integrable random variable, and let $\mathcal{G} \subset \mathcal{F}$ be a sub- σ -algebra. A conditional expectation of X given $\mathcal{G}$ is an integrable, $\mathcal{G}$ -measurable random variable $\mathbb{E}[X \mid \mathcal{G}]$ such that, for every $G \in \mathcal{G}$,

$$\int_G X \, d\mathbb{P} = \int_G \mathbb{E}[X \mid \mathcal{G}] \, d\mathbb{P}.$$

For vector-valued X , this is understood coordinatewise. When $\mathcal{G} = \sigma(Y)$, we write $\mathbb{E}[X \mid Y]$ for $\mathbb{E}[X \mid \sigma(Y)]$.

Remark 2.1.19. The conditional expectation exists and is unique up to redefinition on a $\mathbb{P}$ -null set. Equivalently, the defining identity may be tested against every bounded $\mathcal{G}$ -measurable function.

Lemma 2.1.20 (Tower property (Oksendal, 1998, Theorem B.3)). *Let $(\Omega, \mathcal{F}, \mathbb{P})$ be a probability space, let $\mathcal{H} \subseteq \mathcal{G} \subseteq \mathcal{F}$ be sub- σ -algebras, and let $X : (\Omega, \mathcal{F}) \rightarrow (\mathbb{R}^d, \mathcal{B}(\mathbb{R}^d))$ be integrable. Then*

$$\mathbb{E}[\mathbb{E}[X \mid \mathcal{G}] \mid \mathcal{H}] = \mathbb{E}[X \mid \mathcal{H}], \quad \mathbb{P}\text{-almost surely.}$$

Definition 2.1.21 (Conditional law). Let X and Y be random variables taking values in Polish spaces S_X and S_Y . The conditional law of X given $Y = y$ is denoted by $\text{Law}(X \in dx \mid Y = y)$ and is characterized by

$$\text{Law}(X \in A \mid Y = y) := \mathbb{P}(X \in A \mid Y = y) := \mathbb{E}[\mathbf{1}_A(X) \mid Y = y], \quad A \in \mathcal{B}(S_X).$$

For Polish spaces this conditional law exists (Bogachev, 2007, Ex. 10.7.5). For every integrable Borel function $\varphi : S_X \rightarrow \mathbb{R}$, it satisfies

$$\mathbb{E}[\varphi(X) \mid Y = y] = \int \varphi(x) \, \text{Law}(X \in dx \mid Y = y).$$

A central notion in probability theory is that of independence, which we define for events, σ -algebras and random variables.

Definition 2.1.22. Let $(\Omega, \mathcal{F}, \mathbb{P})$ be a probability space. A collection of events $\{A_i\}_{i \in I} \subset \mathcal{F}$ is *independent* if for every finite subset $J \subset I$,

$$\mathbb{P}\left(\bigcap_{i \in J} A_i\right) = \prod_{i \in J} \mathbb{P}(A_i).$$

Definition 2.1.23. Let $(\Omega, \mathcal{F}, \mathbb{P})$ be a probability space. A collection of sub- σ -algebras $\{\mathcal{G}_i\}_{i \in I}$ of $\mathcal{F}$ is *independent* if for every finite subset $J \subset I$ and every choice of $A_i \in \mathcal{G}_i$ for $i \in J$,

$$\mathbb{P}\left(\bigcap_{i \in J} A_i\right) = \prod_{i \in J} \mathbb{P}(A_i).$$

Definition 2.1.24. Let $(\Omega, \mathcal{F}, \mathbb{P})$ be a probability space. A collection of random variables $\{X_i\}_{i \in I}$ is *independent* if the generated σ -algebras $\{\sigma(X_i)\}_{i \in I}$ are independent. A random variable X is said to be *independent of* a sub- σ -algebra $\mathcal{G} \subset \mathcal{F}$ if $\sigma(X)$ and $\mathcal{G}$ are independent.

2.2 STOCHASTIC PROCESSES, BROWNIAN MOTION, AND ITÔ INTEGRATION

Definition 2.2.1. Let $(\Omega, \mathcal{F}, \mathbb{P})$ be a probability space and $I \subseteq \mathbb{R}$ a time interval. A stochastic process $\{X_t\}_{t \in I}$ on a Polish space S is a Borel-measurable mapping $X : I \times \Omega \rightarrow S$, where $I \times \Omega$ is equipped with the product σ -algebra $\mathcal{B}(I) \otimes \mathcal{F}$ (cf. Definition 2.1.6).

Remark 2.2.2. Common choices are $I = [0, T]$, $I = [0, \infty)$ or $I = [S, T]$. We write $\{X_t\}$ to de-emphasize the particular time-indexing set, when appropriate.

Definition 2.2.3. Let $(\Omega, \mathcal{F}, \mathbb{P})$ be a probability space. A stochastic process $\{X_t\}$ is said to have continuous sample paths if the mapping $t \rightarrow X_t(\omega)$ is continuous for $\mathbb{P}$ -almost every ω .

Definition 2.2.4. A filtration $\{\mathcal{F}_t\}_{t \in I}$, $I \subset \mathbb{R}$, on the probability space $(\Omega, \mathcal{F}, \mathbb{P})$ is a collection of σ -algebras such that $\mathcal{F}_s \subset \mathcal{F}_t$ for $s \leq t$. A filtered probability space is the quadruple $(\Omega, \mathcal{F}, \{\mathcal{F}_t\}_{t \in I}, \mathbb{P})$.

Remark 2.2.5. Again, for brevity of notation, we write $\{\mathcal{F}_t\}$.

Definition 2.2.6. Let $(\Omega, \mathcal{F}, \{\mathcal{F}_t\}, \mathbb{P})$ be a filtered probability space. A stochastic process $\{X_t\}$ on the filtered probability space $(\Omega, \mathcal{F}, \{\mathcal{F}_t\}, \mathbb{P})$ is said to be adapted to the filtration $\{\mathcal{F}_t\}$ if $\omega \rightarrow X_t(\omega)$ is $\mathcal{F}_t$ measurable, for each t .

Definition 2.2.7. Let $(\Omega, \mathcal{F}, \mathbb{P})$ be a probability space, $\mathcal{G} \subseteq \mathcal{F}$ a sub- σ -algebra, and $B \in \mathcal{F}$. The *conditional probability* of B given $\mathcal{G}$ is the random variable $\mathbb{P}(B \mid \mathcal{G}) := \mathbb{E}[I_B \mid \mathcal{G}]$.

Definition 2.2.8. Let $(\Omega, \mathcal{F}, \{\mathcal{F}_t\}, \mathbb{P})$ be a filtered probability space. An adapted stochastic process $\{X_t\}$ on a Polish space S is a *Markov process* if, for every $A \in \mathcal{B}(S)$ and all $s \leq t$, for $\mathbb{P}$ -almost every ω ,

$$\mathbb{P}(X_t \in A \mid \mathcal{F}_s)(\omega) = \mathbb{P}(X_t \in A \mid X_s = X_s(\omega)),$$

where the right-hand side is understood through Definition 2.1.21, applied to X_t and $Y = X_s$.

That is, for a Markov process, the dynamics of the process depend only on the present state X_s . Any further information contained in $\mathcal{F}_s$ beyond X_s has no effect.

Corollary 2.2.9. An adapted stochastic process $\{X_t\}$ on S is a Markov process if and only if, for every bounded measurable $g : S \rightarrow \mathbb{R}$ and all $s \leq t$,

$$\mathbb{E}[g(X_t) \mid \mathcal{F}_s] = \mathbb{E}[g(X_t) \mid X_s], \quad \mathbb{P}\text{-almost surely.} \quad (1)$$

Proof. First, taking $g = I_A$ in (1) for $A \in \mathcal{B}(S)$ recovers Definition 2.2.8, as $\mathbb{E}[I_A(X_t) \mid \mathcal{G}] = \mathbb{P}(X_t \in A \mid \mathcal{G})$. Conversely, the identity for indicator functions extends to simple functions by linearity, and then to all bounded measurable g through a simple function approximation argument and Lebesgue's monotone convergence theorem. $\square$

Remark 2.2.10. If the reader is not familiar with arguments involving simple function approximations and Lebesgue's monotone convergence theorem, they can consult Rudin (1987) or any other text on measure theory.

211 **Corollary 2.2.11** (Markov tower property). *Let $\{X_t\}$ be a Markov process on S . Then for every bounded*
 212 *measurable $g: S \rightarrow \mathbb{R}$ and all $s \leq u \leq t$,*

$$\mathbb{E}[g(X_t) \mid X_s] = \mathbb{E}[\mathbb{E}[g(X_t) \mid X_u] \mid X_s], \quad \mathbb{P}\text{-almost surely.} \quad (2)$$

213 *Proof.* Since $\mathcal{F}_s \subset \mathcal{F}_u$ (by the defining property of a filtration), the tower property of conditional
 214 expectation gives

$$\mathbb{E}[g(X_t) \mid X_s] = \mathbb{E}[g(X_t) \mid \mathcal{F}_s] = \mathbb{E}[\mathbb{E}[g(X_t) \mid \mathcal{F}_u] \mid \mathcal{F}_s] = \mathbb{E}[\mathbb{E}[g(X_t) \mid X_u] \mid \mathcal{F}_s],$$

215 where the first and second equalities use Corollary 2.2.9 and the tower property, respectively. Now, by the
 216 Doob–Dynkin lemma, write the inner conditional expectation as $h(X_u)$, where $h(x) := \mathbb{E}[g(X_t) \mid X_u = x]$
 217 in the sense of Definition 2.1.21. Applying Corollary 2.2.9 once more, we get

$$\mathbb{E}[h(X_u) \mid \mathcal{F}_s] = \mathbb{E}[h(X_u) \mid X_s] = \mathbb{E}[\mathbb{E}[g(X_t) \mid X_u] \mid X_s],$$

218 which gives (2). □

219 Now we consider $S = \mathbb{R}$. Arguably the most important stochastic process is Brownian motion,
 220 which was initially intended to model the motion of a heavy particle suspended in a fluid, also known
 221 as a Brownian particle. Through physical observations, Einstein (1905) characterized the motion of a
 222 Brownian particle in a way consistent with the modern mathematical definition of Brownian motion,
 223 however he did not provide a mathematical proof of its existence. This was shown by Wiener (1923) and
 224 Brownian motion is therefore sometimes called a Wiener process.

225 **Definition 2.2.12.** Brownian motion $\{B_t\}_{t \geq 0}$ is a real-valued stochastic process with continuous sample
 226 paths on a filtered probability space $(\Omega, \mathcal{F}, \{\mathcal{F}_t\}_{t \geq 0}, \mathbb{P})$, adapted to $\{\mathcal{F}_t\}_{t \geq 0}$, satisfying

- 227 1. $B_0 = 0$, $\mathbb{P}$ -almost surely.
- 228 2. For all $0 \leq s \leq t$, the increment $B_t - B_s$ is independent of $\mathcal{F}_s$.
- 229 3. For all $0 \leq s \leq t$, $B_t - B_s \sim \mathcal{N}(0, t - s)$.

230 **Remark 2.2.13.** Brownian motion, together with its filtration, can be shown to exist with the Kol-
 231 mogorov extension and continuity theorems (cf. (Øksendal, 1998, Ch. 2)). It can also be useful to relax
 232 the condition that $B_0 = 0$ and consider arbitrary starting points, instead.

233 **Definition 2.2.14.** A d -dimensional Brownian motion is $B_t = (B_t^1, \dots, B_t^d)$ with each component inde-
 234 pendent. For further details regarding the construction and various properties of d -dimensional Brownian
 235 motion, see e.g. (Øksendal, 1998, Ch. 2).

236 Now we come to the key definition of the Itô integral and motivate stochastic differential equations.
 237 To attempt to extend the theory of ordinary differential equations to include a stochastic term, one may
 238 start with

$$\frac{dX_t}{dt} = b(t, X_t) + \sigma(t, X_t) \cdot \xi_t \quad (3)$$

239 where ξ_t is some stochastic noising term, white noise. It is in fact in such a way that Langevin (1908)
 240 proposed an alternative characterization for the motion of a Brownian particle - reasoning from Newton's
 241 second law that the velocity of the particle (X_t in the above) was subject to random perturbations from
 242 collisions with the surrounding particles Koski (2017). Mathematically though, there is no stochastic
 243 process ξ_t with sufficiently nice properties (cf. Ch. 3 Øksendal (1998) for a discussion).

244 Instead, let $0 = t_0 < \dots < t_K = T$. Discretizing (3), we have

$$X_{k+1} = X_k + b(t_k, X_k)\Delta t + \sigma(t_k, X_k)\xi_k\Delta t.$$

245 Replacing $\xi_k\Delta t_k$ with the increment $\Delta V_k = V_{t_{k+1}} - V_{t_k}$ of an actual stochastic process $\{V_t\}_{t \geq 0}$, the only
 246 one with sufficiently nice properties in this model (continuous sample paths, stationarity: $V_t \stackrel{d}{=} V_0$ for all
 247 $t \geq 0$, independent increments with mean zero) turns out to be Brownian motion (Øksendal, 1998, Ch.
 248 3). So we obtain

$$X_k = X_0 + \sum_{j=0}^{k-1} b(t_j, X_j)\Delta t_j + \sum_{j=0}^{k-1} \sigma(t_j, X_j)\Delta B_j.$$

249 The *Itô integral* passes the second sum to an (appropriately taken) continuous limit. We follow
 250 (Øksendal, 1998, Ch. 3), here.

Definition 2.2.15 ((Øksendal, 1998, Definition 3.1.4)). Let $\{B_t\}$ be a Brownian motion on a filtered probability space $(\Omega, \mathcal{F}, \{\mathcal{F}_t\}, \mathbb{P})$, and let $0 \leq S < T$. Denote by $\mathcal{V}(S, T)$ the class of functions $f: [S, T] \times \Omega \rightarrow \mathbb{R}$ such that

i) $f: [S, T] \times \Omega \rightarrow \mathbb{R}$ is a stochastic process that is adapted to the filtration $\{\mathcal{F}_t\}_{S \leq t \leq T}$, in the sense of Definitions 2.2.1 and 2.2.6.

ii) $\mathbb{E}[\int_S^T f(t, \omega)^2 dt] < \infty$.

A function $\phi \in \mathcal{V}(S, T)$ is *elementary* if it is piecewise constant in time, $\phi(t, \omega) = \sum_j e_j(\omega) I_{[t_j, t_{j+1})}(t)$ for a partition $S = t_0 < \dots < t_K = T$ with each e_j being $\mathcal{F}_{t_j}$ -measurable. For each such ϕ , the Itô integral is defined as

$$\int_S^T \phi(t, \omega) dB_t(\omega) := \sum_j e_j(\omega) (B_{t_{j+1}} - B_{t_j}).$$

The integral is then extended to all of $\mathcal{V}(S, T)$ by an L^2 approximation argument.

Definition 2.2.16 ((Øksendal, 1998, Definition 3.1.6)). The *Itô integral* of $f \in \mathcal{V}(S, T)$ is the $L^2(\mathbb{P})$ -limit

$$\int_S^T f(t, \omega) dB_t(\omega) := \lim_{n \rightarrow \infty} \int_S^T \phi_n(t, \omega) dB_t(\omega),$$

where $\{\phi_n\}$ is any sequence of elementary functions in $\mathcal{V}(S, T)$ with $\mathbb{E}[\int_S^T (f - \phi_n)^2 dt] \rightarrow 0$. In particular, the limit exists and does not depend on the approximating sequence.

The scalar construction above can then be extended to integrating a matrix-valued stochastic process against a vector-valued Brownian motion.

Definition 2.2.17 ((Øksendal, 1998, Definition 3.3.1)). Let $\{B_t\} = (B_t^1, \dots, B_t^n)$ be an n -dimensional Brownian motion, and let $\mathcal{V}^{m \times n}(S, T)$ be the class of $m \times n$ matrix-valued functions $v = [v_{ij}]$ with each entry $v_{ij} \in \mathcal{V}(S, T)$. The Itô integral of $v \in \mathcal{V}^{m \times n}(S, T)$ is the $\mathbb{R}^m$ -valued random variable (column vector) whose i -th component is

$$\left(\int_S^T v dB_t \right)_i := \sum_{j=1}^n \int_S^T v_{ij}(t, \omega) dB_t^j(\omega), \quad i = 1, \dots, m,$$

where each summand is the one-dimensional Itô integral of Definition 2.2.16.

Remark 2.2.18. A frequent special case for generative modelling is an $\mathbb{R}$ -valued diffusion coefficient $\sigma_t \in \mathcal{V}(S, T)$ integrated against an n -dimensional Brownian motion $\{B_t\}$. This is a special case of Definition 2.2.17 with the matrix-valued integrand $\sigma_t I_{n \times n} \in \mathcal{V}^{n \times n}(S, T)$, where $I_{n \times n}$ is the identity matrix, so that

$$\int_S^T \sigma_t dB_t := \int_S^T \sigma_t I_{n \times n} dB_t, \quad \left(\int_S^T \sigma_t dB_t \right)_i = \int_S^T \sigma_t dB_t^i.$$

Remark 2.2.19. First, it holds that $B_t \sim \mathcal{N}(0, t)$ (due to the characterizing property $B_t - B_s \sim \mathcal{N}(0, t - s)$). But also in fact, the Itô integral of a deterministic function $t \mapsto f(t)$ in the class $\mathcal{V}(S, T)$ is also Gaussian:

$$\int_S^T f(t) dB_t \sim \mathcal{N}(0, \sigma^2), \quad \sigma^2 = \int_S^T f(t)^2 dt.$$

This will prove useful later on, as it means that sometimes one can simply generate Gaussian random numbers instead of simulating an Itô integral numerically (given that $\int_S^T f(t)^2 dt$ is known *a priori*).

With the Itô integral in hand, we can make precise the notion of a stochastic differential equation.

Definition 2.2.20 ((Øksendal, 1998, Ch. 5)). Let $b: [0, T] \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ and $\sigma: [0, T] \times \mathbb{R}^n \rightarrow \mathbb{R}^{n \times m}$ be Borel-measurable, and let $\{B_t\}$ be an m -dimensional Brownian motion. A stochastic differential equation (SDE) is an expression of the form

$$dX_t = b(t, X_t) dt + \sigma(t, X_t) dB_t, \quad X_0 = Z, \quad (4)$$

with b the *drift*, σ the *diffusion coefficient*, and Z the (possibly random) *initial condition* X_0 . A strong solution of this SDE is a stochastic process $\{X_t\}_{0 \leq t \leq T}$ adapted to the filtration generated by Z and $\{B_s\}_{0 \leq s \leq t}$, satisfying the corresponding Itô integral equation

$$X_t = Z + \int_0^t b(s, X_s) ds + \int_0^t \sigma(s, X_s) dB_s, \quad 0 \leq t \leq T. \quad (5)$$

In the above, the last term is the multidimensional Itô integral in the sense of Definition 2.2.17. The differential form in (4) is (by definition) shorthand for the integral form in (5).

Under linear growth and Lipschitz conditions on the drift and diffusion coefficients, we have existence and uniqueness of solutions:

Theorem 2.2.21 (Øksendal, 1998, Theorem 5.2.1). *Let $T > 0$ and let $b: [0, T] \times \mathbb{R}^n \rightarrow \mathbb{R}^n$, $\sigma: [0, T] \times \mathbb{R}^n \rightarrow \mathbb{R}^{n \times m}$ be measurable and satisfy that for some $C, D > 0$ and for all $x, y \in \mathbb{R}^n$ and $t \in [0, T]$, it holds:*

$$|b(t, x)| + |\sigma(t, x)| \leq C(1 + |x|), \quad |b(t, x) - b(t, y)| + |\sigma(t, x) - \sigma(t, y)| \leq D|x - y|.$$

In the above, $|\sigma|^2 = \sum_{i,j} |\sigma_{ij}|^2$. Let Z be a random variable independent of the σ -algebra generated by (the collection of functions) $\{B_t\}_{t \geq 0}$, with $\mathbb{E}[|Z|^2] < \infty$. Then the SDE (4) has a unique solution $\{X_t\}_{0 \leq t \leq T}$ with continuous sample paths adapted to the filtration generated by Z and $\{B_s\}_{s \leq t}$, satisfying $\mathbb{E}[\int_0^T |X_t|^2 dt] < \infty$. Here uniqueness means any two such solutions agree for all $0 \leq t \leq T$, $\mathbb{P}$ -almost surely.

We state Theorem 2.2.21 for context and completeness, but will not really make use of it. The SDEs appearing in generative modelling applications have coefficients for which existence and uniqueness is either standard or simply assumed *ad hoc*.

2.3 EULER–MARUYAMA DISCRETIZATION

For SDEs such as (4) to be used as practical sampling algorithms, one needs a numerical algorithm for approximating their paths. The simplest such algorithm is Euler–Maruyama. On a grid $0 = t_0 < t_1 < \dots < t_L = T$ with constant step size $h = T/L$, the integral form (5) is approximated by replacing the drift integral with a left-endpoint Riemann sum and the Brownian increment by $B_{t_{k+1}} - B_{t_k} \sim \mathcal{N}(0, hI_m)$. Thus, for an SDE

$$dX_t = b(t, X_t) dt + \sigma(t, X_t) dB_t$$

on $\mathbb{R}^n$, Euler–Maruyama sets $\bar{X}_0 = Z$ and iterates

$$\bar{X}_{k+1} = \bar{X}_k + b(t_k, \bar{X}_k) h + \sqrt{h} \sigma(t_k, \bar{X}_k) \xi_k, \quad \xi_k \sim \mathcal{N}(0, I_m). \quad (6)$$

The convergence properties of Euler–Maruyama are outside the scope of this project, but one may refer to Talay and Tubaro (1990); Kloeden and Platen (1992).

2.4 FELLER PROCESSES, INFINITESIMAL GENERATORS

This section introduces Feller processes and infinitesimal generators, following Holderrieth et al. (2025) and Ethier and Kurtz (1986). The point is to describe diffusions, jump processes, and related Markov processes on locally compact Polish spaces using the same notation. This viewpoint becomes useful as it e.g. can provide a unified perspective for discrete diffusion models Lou et al. (2024) and diffusion models on Riemannian manifolds De Bortoli et al. (2022).

Let S be the state space, which we take to be a locally compact Polish space (Definition 2.1.10), and let $C_0(S)$ be the space of continuous functions on S vanishing at infinity, equipped with the supremum norm $\|\cdot\|_\infty$.

Definition 2.4.1. Let $(\Omega, \mathcal{F}, \{\mathcal{F}_t\}, \mathbb{P})$ be a filtered probability space carrying a Markov process $\{X_t\}_{0 \leq t \leq T}$ on S , let $g: (\Omega, \mathcal{F}) \rightarrow (\mathbb{R}^d, \mathcal{B}(\mathbb{R}^d))$ be an integrable random variable, and fix $0 \leq s \leq T$. We write

$$\mathbb{E}_{s,x}[g] := \mathbb{E}[g \mid X_s = x].$$

In the time-homogeneous case we abbreviate $\mathbb{E}_x := \mathbb{E}_{0,x}$, i.e. the expectation for the process started at $X_0 = x$.

Remark 2.4.2. For the remainder of the section, we assume $\{X_t\}$ is a Markov process on a filtered probability space $(\Omega, \mathcal{F}, \{\mathcal{F}_t\}, \mathbb{P})$, adapted to the filtration.

Definition 2.4.3. The transition operators $P_{s,t}$ associated to $\{X_t\}$ act on bounded measurable test functions $f : (S, \mathcal{B}(S)) \rightarrow (\mathbb{R}, \mathcal{B}(\mathbb{R}))$ through

$$P_{s,t}f(x) := \mathbb{E}_{s,x}[f(X_t)], \quad 0 \leq s \leq t \leq T.$$

By the tower and Markov properties (cf. Corollary 2.2.11), for $s \leq u \leq t$,

$$P_{s,u}P_{u,t}f(x) = \mathbb{E}[\mathbb{E}[f(X_t) \mid X_u] \mid X_s = x] = \mathbb{E}[f(X_t) \mid X_s = x] = P_{s,t}f(x),$$

so that $P_{s,t} = P_{s,u}P_{u,t}$ for $s \leq u \leq t$. This is known as the Chapman-Kolmogorov equation.

Definition 2.4.4. A Markov process is *time-homogeneous* if its transition operators depend only on the elapsed time, i.e. $P_{s,t} = P_{0,t-s}$ for all $s \leq t$. In that case we write $T_t := P_{0,t}$, and $\{T_t\}_{t \geq 0}$. This forms a one-parameter semigroup

$$T_0 = \text{Id}, \quad T_{s+t} = T_s T_t.$$

Definition 2.4.5. A *bounded linear operator* on $C_0(S)$ is a linear map $T : C_0(S) \rightarrow C_0(S)$ whose operator norm

$$\|T\| := \sup_{f \neq 0} \frac{\|Tf\|_\infty}{\|f\|_\infty}.$$

Definition 2.4.6 ((Ethier and Kurtz, 1986, Ch. 1)). A *strongly continuous contraction semigroup* on $C_0(S)$ is a family $\{T_t\}_{t \geq 0}$ of bounded linear operators on $C_0(S)$ (Definition 2.4.5) such that

$$T_0 = \text{Id}, \quad T_{s+t} = T_s T_t \quad (s, t \geq 0),$$

each T_t is a contraction, $\|T_t f\|_\infty \leq \|f\|_\infty$, and $t \mapsto T_t f$ is strongly continuous, i.e. $\|T_t f - f\|_\infty \rightarrow 0$ as $t \downarrow 0$, for every $f \in C_0(S)$.

Definition 2.4.7 ((Ethier and Kurtz, 1986, Ch. 4)). A *Feller semigroup* is a strongly continuous contraction semigroup $\{T_t\}_{t \geq 0}$ on $C_0(S)$ (Definition 2.4.6) that moreover satisfies

$$f \geq 0 \implies T_t f \geq 0, \quad T_t(C_0(S)) \subseteq C_0(S).$$

A time-homogeneous Markov process is a *Feller process* if its semigroup $\{T_t\}$ is a Feller semigroup.

Remark 2.4.8. For a semigroup $\{T_t\}$ coming from the transition operators of a time-homogeneous Markov process, the positivity condition $T_t f(x) = \mathbb{E}_{0,x}[f(X_t)] \geq 0$ when $f \geq 0$ is automatic.

Definition 2.4.9 ((Ethier and Kurtz, 1986, Ch. 1)). The *infinitesimal generator* of a Feller semigroup $\{T_t\}$ is the operator

$$\mathcal{L}f := \lim_{h \downarrow 0} \frac{T_h f - f}{h},$$

defined on the domain $\mathcal{D}(\mathcal{L}) \subseteq C_0(S)$ of those f for which the limit exists in $C_0(S)$, i.e. with respect to $\|\cdot\|_\infty$.

More generally, in the time-inhomogeneous setting relevant to generative modelling, the generator carries a time index and is defined directly from the transition operators $P_{s,t}$ through the supremum-norm limits

$$\mathcal{D}(\mathcal{L}_t) := \{f \in C_0(S) : \lim_{h \downarrow 0} \frac{P_{t,t+h}f - f}{h} \text{ exists in } (C_0(S), \|\cdot\|_\infty)\}, \quad \mathcal{L}_t f := \lim_{h \downarrow 0} \frac{P_{t,t+h}f - f}{h}.$$

One way to reduce the time-inhomogeneous theory to the time-homogeneous as in Holderrieth et al. (2025) is by extending the state space to include time. On the extended state space $\mathbb{R} \times S$, consider the process $\tilde{X}_t = (t, X_t)$.

Its time component evolves deterministically at unit speed, $dt/dt = 1$, while its spatial component evolves under $\mathcal{L}_t$, so $\{\tilde{X}_t\}$ is a time-homogeneous Markov process. On the extended state space, its generator is

$$\mathcal{L}f(t, x) = \partial_t f(t, x) + \mathcal{L}_t f^t(x),$$

where the ∂_t term comes from the unit-speed drift of the time coordinate and $\mathcal{L}_t$ acts on the spatial variable. Consequently, the time-homogeneous theory applies to $\{\tilde{X}_t\}$ through the homogeneous generator $\mathcal{L} = \partial_t + \mathcal{L}_t$ acting on test functions $(t, x) \mapsto f(t, x)$ in a suitable domain, and the resulting statements transfer back to the original time-inhomogeneous process $\{X_t\}$. See, for example, (Holderrieth et al., 2025, App. A) for details.

Theorem 2.4.10 (Forward and backward equations, (Ethier and Kurtz, 1986, Ch. 1, Proposition 1.5)).
 Let $\{T_t\}_{t \geq 0}$ be a strongly continuous contraction semigroup on $C_0(S)$ (Definition 2.4.6) with generator $\mathcal{L}$. Then for every $f \in \mathcal{D}(\mathcal{L})$ and $t \geq 0$ one has $T_t f \in \mathcal{D}(\mathcal{L})$ and

$$\frac{d}{dt} T_t f = \mathcal{L} T_t f = T_t \mathcal{L} f.$$

Corollary 2.4.11. Let $\{X_t\}$ be a Feller process with semigroup $\{T_t\}$ and generator $\mathcal{L}$. Since a Feller semigroup is in particular a strongly continuous contraction semigroup (Definition 2.4.7), Theorem 2.4.10 applies. For every $f \in \mathcal{D}(\mathcal{L})$, its forward equation $\frac{d}{dt} T_t f = T_t \mathcal{L} f$ reads

$$\frac{d}{dt} \mathbb{E}_x[f(X_t)] = \mathbb{E}_x[\mathcal{L} f(X_t)].$$

This is the Kolmogorov Forward Equation (KFE).

Remark 2.4.12. In the generative modelling literature (see e.g. Holderrieth et al. (2025)), the conditions for differentiating under integrals (see e.g. Rudin (1987) or other texts on measure theory for such conditions) are assumed *ad hoc*.

Corollary 2.4.13 (Time-inhomogeneous KFE). Let $f \in \mathcal{D}(\mathcal{L}_t)$, for each t . Applying Corollary 2.4.11 to the time-homogenized process $\tilde{X}_t = (t, X_t)$ and the test function $\tilde{f}(t, x) := f(x)$, gives the time-inhomogeneous Kolmogorov Forward Equation (KFE)

$$\frac{d}{dt} \mathbb{E}_{s,x}[f(X_t)] = \mathbb{E}_{s,x}[\mathcal{L}_t f(X_t)], \quad f \in \mathcal{D}(\mathcal{L}_t).$$

If $p_t = \text{Law}(X_t)$, averaging over the starting point and differentiating under the integral yields

$$\frac{d}{dt} \langle p_t, f \rangle = \langle p_t, \mathcal{L}_t f \rangle. \quad (7)$$

Concretely, we adopt the following assumptions, which we take from Holderrieth et al. (2025, App. A.2).

Assumption 2.4.14 ((Holderrieth et al., 2025, App. A.2)). Throughout, the Markov processes and probability paths under consideration are assumed to satisfy:

- i) $\{X_t\}$ is a Feller process, in the sense that the time-homogenized process $\tilde{X}_t = (t, X_t)$ has a Feller semigroup (Definition 2.4.7).
- ii) On every time interval $[s, t]$, the expected number of discontinuities of $\tau \mapsto X_\tau$ is finite.
- iii) There is a subspace $\mathcal{T} \subseteq C_0(S)$, dense in $C_0(S)$, with $\mathcal{T} \subseteq \mathcal{D}(\mathcal{L}_t)$ and $t \mapsto \mathcal{L}_t f$ continuous for every $f \in \mathcal{T}$, and such that $\mathcal{T}$ separates measures: probability measures μ_1, μ_2 on S are equal if and only if $\langle \mu_1, f \rangle = \langle \mu_2, f \rangle$ for all $f \in \mathcal{T}$.
- iv) Every probability path $(p_t)_{0 \leq t \leq 1}$ considered has $t \mapsto \langle p_t, f \rangle$ continuous for all $f \in \mathcal{T}$.
- v) If $X_0 \sim p_0$ and $\partial_t \langle p_t, f \rangle = \langle p_t, \mathcal{L}_t f \rangle$ holds for all $f \in \mathcal{T}$, then $X_t \sim p_t$ for all $0 \leq t \leq 1$.

Remark 2.4.15. See (Holderrieth et al., 2025, App. A.2) for further discussion of conditions, in particular regarding the validity of the fifth assumption on various state spaces – however, they have not guaranteed or provided conditions under which these assumptions hold for the generator resulting from training a neural network. But it would be an interesting line of research to propose appropriate constraints and pin down conditions under which these conditions hold for the generator resulting from training.

2.5 JUMP PROCESSES AND EXAMPLES OF GENERATORS

This formalism also allows the description of processes with jumps. The prototypical such process is the Poisson process, which counts events that arrive independently at a constant rate.

Definition 2.5.1 ((Ethier and Kurtz, 1986, Ch. 4)). A *Poisson process* with rate $\lambda > 0$ is an integer-valued process $\{N_t\}_{t \geq 0}$ with $N_0 = 0$, independent increments, and $N_t - N_s \sim \text{Poisson}(\lambda(t - s))$ for all $0 \leq s \leq t$. Equivalently, it starts at $N_0 = 0$ and increases by jumps of size one, where the holding times between successive jumps are independent and exponentially distributed with rate λ , so that

$$N_{t+\Delta t} = \begin{cases} N_t & \text{with probability } 1 - \lambda\Delta t + o(\Delta t), \\ N_t + 1 & \text{with probability } \lambda\Delta t + o(\Delta t). \end{cases}$$

This extends to jump processes defined on broader state spaces, as well, which we see in the following:

Definition 2.5.2 ((Holderrieth et al., 2025)). A *pure jump process* on S is determined by a *jump measure* $Q_t(dy; x)$, where for each t and $x \in S$ the measure $Q_t(\cdot; x)$ is a finite measure on $S \setminus \{x\}$ (Holderrieth et al., 2025), so that no mass sits at the current state x . Its generator is

$$\mathcal{L}_t f(x) = \int_S (f(y) - f(x)) Q_t(dy; x).$$

From this, one recovers the *jump rate* $\lambda_t(x)$ and *jump-destination law* $J_t(dy; x)$ via

$$\lambda_t(x) := \int_S Q_t(dy; x), \quad J_t(dy; x) := \frac{Q_t(dy; x)}{\lambda_t(x)}.$$

In the above, $J_t(dy; x)$ is a probability measure on S whenever $\lambda_t(x) > 0$, and otherwise it is immaterial, as the rate of jumping is zero. This gives the law of the location jumped to from x .

Remark 2.5.3. In the time-homogeneous case, from a state $x \in S$, the process is static for an exponentially distributed waiting time of rate $\lambda(x)$, and then jumps to $y \sim J(dy; x)$. In the time-inhomogeneous case, $\lambda_t(x)$ is instead an instantaneous hazard. Concretely, as stated in Holderrieth et al. (2025), a process $\{X_t\}$ with generator as in Definition 2.5.2 evolves infinitesimally according to

$$X_{t+\Delta t} = \begin{cases} X_t & \text{with probability } 1 - \lambda_t(x)\Delta t + o(\Delta t), \\ \sim J_t(dy; x) & \text{with probability } \lambda_t(x)\Delta t + o(\Delta t). \end{cases}$$

Remark 2.5.4. For jump processes with further conditions, see (Ethier and Kurtz, 1986, Ch. 4). Conditions for when our assumptions (Assumption 2.4.14) originating in Holderrieth et al. (2025) are satisfied for such jump processes are not elucidated, but rather assumed.

On $S = \mathbb{R}^d$, a diffusion solving an SDE of the form (4) instead has a second-order differential generator.

Definition 2.5.5 ((Øksendal, 1998, Theorem 7.3.3)). A diffusion on $\mathbb{R}^d$ solving $dX_t = b_t(X_t)dt + \sigma_t(X_t)dB_t$, with *diffusion matrix* $a_t(x) := \sigma_t(x)\sigma_t(x)^\top$, has time-inhomogeneous infinitesimal generator

$$\mathcal{L}_t f(x) = b_t(x) \cdot \nabla f(x) + \frac{1}{2} a_t(x) : \nabla^2 f(x)$$

where $\nabla^2 f$ is the Hessian and $A : B = \sum_{i,j} A_{ij} B_{ij}$ is the Frobenius inner product.

Remark 2.5.6. Both the jump generator (Definition 2.5.2) and the diffusion generator (Definition 2.5.5) are understood to act on a class of test functions $\mathcal{T}$ in the sense of Holderrieth et al. (2025) (cf. Assumption 2.4.14(iii)). For $S = \mathbb{R}^d$ a natural choice is $\mathcal{T} \subset C_0^2(\mathbb{R}^d)$. The precise conditions that such a class and the jump measure Q_t and coefficients b_t, σ_t must satisfy for these expressions to define a genuine Feller generator are not pinned down in Holderrieth et al. (2025). They point to other literature such as Ethier and Kurtz (1986); Stroock and Varadhan (1979) for further details.

Likely, this is again in part due to difficulties finding conditions that are sufficiently adaptive to all the contexts that can arise when training a neural network, and would be an interesting line of research.

Remarkably, on $S = \mathbb{R}^d$ the generators of Markov processes satisfying our relevant conditions admit a common decomposition. The following theorem is stated in Holderrieth et al. (2025). It is related to the work of Courrège (1965–1966).

Theorem 2.5.7 (Universal characterization of generators, (Holderrieth et al., 2025, Theorem 1)). Under our assumptions (Assumption 2.4.14), for the Euclidean state space $S = \mathbb{R}^d$, the generator $\mathcal{L}_t$ of $\{X_t\}_{0 \leq t \leq 1}$ decomposes as

$$\mathcal{L}_t f(x) = \nabla f(x)^\top u_t(x) + \frac{1}{2} \sigma_t^2(x) : \nabla^2 f(x) + \int_{\mathbb{R}^d} (f(y) - f(x)) Q_t(dy; x),$$

where $u_t: \mathbb{R}^d \rightarrow \mathbb{R}^d$ is continuous, $\sigma_t^2(x)$ is a positive semi-definite diffusion matrix continuous in x , and, for every x , $Q_t(\cdot; x)$ is a jump measure in the sense of Definition 2.5.2.

Sampling from a generator of the form in Theorem 2.5.7 is achieved by a first-order discretization that, at each step, either takes a flow-and-diffusion step or with probability proportional to the jump intensity performs a jump.

Algorithm 1. Euler sampling for $S = \mathbb{R}^d$ (Holderrieth et al., 2025, Algorithm 2)

Require: velocity u_t , diffusion σ_t , jump measure Q_t , simple prior distribution p_{simple} , number of steps $L \in \mathbb{N}$ and step size $h = 1/L$ small enough that $h \lambda_t(\bar{X}_t) \leq 1$ along the simulated path

1. Initialize $\bar{X}_0 \sim p_{\text{simple}}$.
2. **for** $t \in \{0, h, 2h, \dots, 1-h\}$ **do**
3. Compute jump intensity $\lambda_t(\bar{X}_t) = \int Q_t(dy; \bar{X}_t)$.
4. Draw $m \sim \text{Bernoulli}(h \lambda_t(\bar{X}_t))$.
5. **if** $m = 1$ **then**
6. Draw $\bar{X}_{t+h} \sim Q_t(\cdot; \bar{X}_t) / \lambda_t(\bar{X}_t)$.
7. **else**
8. Draw $\varepsilon_t \sim \mathcal{N}(0, I)$ and set $\bar{X}_{t+h} \leftarrow \bar{X}_t + h u_t(\bar{X}_t) + \sqrt{h} \sigma_t(\bar{X}_t) \varepsilon_t$.
9. **end if**
10. **end for**

return $\bar{X}_1$

Remark 2.5.8. The particular convergence properties of this discretization have not been discussed in Holderrieth et al. (2025), and the step size $h > 0$ is a quantity whose value is determined by experiments, through weighing accuracy versus performance tradeoffs.

2.6 VARIABLE LENGTH BRANCHING AND DELETIONS

Variable-length data is modelled in Branching Flows (Billera et al., 2026c) on the *disjoint-union* state space

$$S_N = \bigsqcup_{1 \leq n \leq N} \mathcal{E}^n = \{(n, x) : 1 \leq n \leq N, x \in \mathcal{E}^n\},$$

where we, for simplicity, set $\mathcal{E} = \mathbb{R}^d$, which we refer to as the *base space* for this work. Writing $x = (x_1, \dots, x_n) \in (\mathbb{R}^d)^n \cong \mathbb{R}^{dn}$, a convenient metric is

$$d_{S_N}((n, x), (m, y)) = \begin{cases} \min\{1, \|x - y\|_{\mathbb{R}^{dn}}\}, & n = m, \\ 1, & n \neq m. \end{cases}$$

This gives the disjoint-union topology and makes S_N locally compact Polish. Differential-topologically, each component has the smooth chart $\varphi_n(n, x_1, \dots, x_n) = (x_1, \dots, x_n)$ onto $\mathbb{R}^{dn}$.

The more general context of a locally compact Polish base space is discussed in (Billera et al., 2026c), and in particular applied to product spaces of the form $\mathcal{E} = S_1 \times S_2$ where S_1 is discrete and S_2 is a Riemannian manifold.

Test functions on S_N must also be read componentwise. We assume that, for each n , there is a sufficiently nice test-function space $\mathcal{T}^{(n)} \subset C_0^2(\mathbb{R}^{dn})$. The corresponding test class on S_N is

$$\mathcal{T}_N := \{f : S_N \rightarrow \mathbb{R} : f(n, x) = f_n(x) \text{ with } f_n \in \mathcal{T}^{(n)} \text{ for each } 1 \leq n \leq N\}.$$

A split from (n, x) evaluates the $(n+1)$ -component f_{n+1} , a deletion evaluates the $(n-1)$ -component f_{n-1} , and the base diffusion on the component $\{n\} \times \mathbb{R}^{dn}$ differentiates only f_n as a function on $\mathbb{R}^{dn}$.

Now we consider our desired form of branching. For a state $(n, x) = (n, x_1, \dots, x_n) \in S_N$, recall the split, for $n < N$ and $1 \leq j \leq n$,

$$\text{split}_j(n, x_1, \dots, x_n) = (n+1, x_1, \dots, x_j, x_j, x_{j+1}, \dots, x_n),$$

which duplicates the j th entry (moving from component n to $n+1$), and the deletion, for $n > 1$ and $1 \leq j \leq n$,

$$\text{del}_j(n, x_1, \dots, x_n) = (n-1, x_1, \dots, x_{j-1}, x_{j+1}, \dots, x_n),$$

which removes it (moving from n to $n-1$). Let $\lambda_{t,j}(n, x)$ and $\mu_{t,j}(n, x)$ denote the per-element split and deletion rates for admissible operations. Splits and deletions, alone, form a pure-jump process (Definition 2.5.2) with jump measure

$$Q_t(dy; (n, x)) = I_{\{n < N\}} \sum_{j=1}^n \lambda_{t,j}(n, x) \delta_{\text{split}_j(n, x)}(dy) + I_{\{n > 1\}} \sum_{j=1}^n \mu_{t,j}(n, x) \delta_{\text{del}_j(n, x)}(dy).$$

The corresponding generator is

$$\begin{aligned} \mathcal{L}_t^{\text{jump}} f(n, x) &= \int_{S_N} (f(y) - f(n, x)) Q_t(dy; (n, x)) \\ &= I_{\{n < N\}} \sum_{j=1}^n (f(\text{split}_j(n, x)) - f(n, x)) \lambda_{t,j}(n, x) \\ &\quad + I_{\{n > 1\}} \sum_{j=1}^n (f(\text{del}_j(n, x)) - f(n, x)) \mu_{t,j}(n, x), \end{aligned}$$

whose jumps move between the components. Following Definition 2.5.2, the total jump intensity Λ_t (which we use instead of λ_t , here) and jump-destination law $J_t(dy; (n, x))$ are recovered as

$$\Lambda_t(n, x) := \int_{S_N} Q_t(dy; (n, x)) = I_{\{n < N\}} \sum_{j=1}^n \lambda_{t,j}(n, x) + I_{\{n > 1\}} \sum_{j=1}^n \mu_{t,j}(n, x),$$

and, whenever $\Lambda_t(n, x) > 0$,

$$\begin{aligned} J_t(dy; (n, x)) &:= \frac{Q_t(dy; (n, x))}{\Lambda_t(n, x)} \\ &= \frac{1}{\Lambda_t(n, x)} (I_{\{n < N\}} \sum_{j=1}^n \lambda_{t,j}(n, x) \delta_{\text{split}_j(n, x)}(dy) + I_{\{n > 1\}} \sum_{j=1}^n \mu_{t,j}(n, x) \delta_{\text{del}_j(n, x)}(dy)). \end{aligned}$$

If $\Lambda_t(n, x) = 0$, the choice of $J_t(\cdot; (n, x))$ is immaterial because no jump occurs. Infinitesimally, from (n, x) the process jumps with instantaneous hazard $\Lambda_t(n, x)$. It then applies an admissible split_j with probability $\lambda_{t,j}(n, x)/\Lambda_t(n, x)$ or an admissible del_j with probability $\mu_{t,j}(n, x)/\Lambda_t(n, x)$. Together with a base generator $\mathcal{L}_t^{\text{base}}$ that evolves the process within each component $\mathcal{E}^n$ between jumps, the full branching generator is

$$\mathcal{L}_t^{\text{branch}} f(n, x) = \mathcal{L}_t^{\text{jump}} f(n, x) + \mathcal{L}_t^{\text{base}} f(n, x).$$

Remark 2.6.1. We stress explicitly that we make *no* attempt to verify that the conditions in Assumption 2.4.14 of Holderrieth et al. (2025) hold for the branching generator $\mathcal{L}_t^{\text{branch}}$ obtained after training a neural network. We simply assume them, driven in part by successful experiments. To our knowledge such a verification has also not been carried out in the comparatively simple Euclidean case $S = \mathbb{R}^d$. In practice, the learned rates, vector fields and diffusion coefficients must be very well behaved for the model to be useful. Heuristic constraints for Euclidean diffusion generative models can be found in Karras et al. (2022). We do not try to characterize such constraints in the branching context.

The practical first order discretization of the branching generator, following Algorithm 1, becomes the following.

Algorithm 2. Euler sampling for the branching generator on S_N

Require: base step (Algorithm 1), split rates $\lambda_{t,j}$, deletion rates $\mu_{t,j}$, simple prior distribution p_{simple} , step size $h > 0$ small enough that $h \Lambda_t(n_t, \bar{X}_t) \leq 1$ along the simulated path

1. Initialize $(n_0, \bar{X}_0) \sim p_{\text{simple}}$.
2. **for** $t \in \{0, h, 2h, \dots, (L-1)h\}$ **do**
3. Total jump intensity $\Lambda_t(n_t, \bar{X}_t) = I_{\{n_t < N\}} \sum_{j=1}^{n_t} \lambda_{t,j}(n_t, \bar{X}_t) + I_{\{n_t > 1\}} \sum_{j=1}^{n_t} \mu_{t,j}(n_t, \bar{X}_t)$.
4. Draw $m \sim \text{Bernoulli}(h \Lambda_t(n_t, \bar{X}_t))$.
5. **if** $m = 1$ **then**
6. Sample among the admissible operations with $\Pr[\text{split}_j] = \lambda_{t,j}(n_t, \bar{X}_t) / \Lambda_t(n_t, \bar{X}_t)$ for $n_t < N$ and $\Pr[\text{del}_j] = \mu_{t,j}(n_t, \bar{X}_t) / \Lambda_t(n_t, \bar{X}_t)$ for $n_t > 1$.
7. Set $(n_{t+h}, \bar{X}_{t+h}) \leftarrow \text{op}_j(n_t, \bar{X}_t)$.
8. **else**
9. Set $\bar{X}_{t+h} \leftarrow$ one base-generator Euler step of $\mathcal{L}_t^{\text{base}}$ via Algorithm 1 (Holderrieth et al., 2025, Algorithm 2), and $n_{t+h} \leftarrow n_t$.
10. **end if**
11. **end for**
- return** $\bar{X}_1$

Remark 2.6.2. We also do not attempt to characterize the convergence properties of this sampling algorithm, in part because this has not even been done in the comparatively simple Euclidean case of Holderrieth et al. (2025).

3 DIFFUSION GENERATIVE MODELS AND LATENT PROCESS GENERATOR MATCHING

We now use the preceding definitions for generative modelling. We first recall stochastic interpolants on $\mathbb{R}^d$ (Albergo et al., 2023). Then we state Latent Process Generator Matching (Billera et al., 2026a), the form used for Branching Flows in §5. The goal is to identify a marginal process that transports samples from the simple prior distribution p_{simple} to the data distribution p_{data} over $[0, 1]$ in the idealized population setting.

3.1 DIFFUSION GENERATIVE MODELS THROUGH STOCHASTIC INTERPOLANTS

Following the stochastic interpolants viewpoint (Albergo et al., 2023), we assume throughout this section that p_{simple} and p_{data} are continuous densities with respect to Lebesgue measure. Note that this places an underlying hypothesis on the hypothetical data distribution p_{data} , of which the dataset one has access to is assumed to be an i.i.d. sample.

We adopt the standard abuse of notation that identifies a density p with the measure $p(x) dx$ it induces. The symbol p denotes a *density* when evaluated at a point, as in $p(x)$, and the corresponding *measure* otherwise, as in the product measure $p_{\text{simple}} \otimes p_{\text{data}}$.

Now, in the stochastic interpolants attempt to solve the generative modelling problem, we seek a diffusion process $\{X_t\}_{0 \leq t \leq 1}$ on $\mathbb{R}^d$

$$dX_t = f_t(X_t) dt + g_t(X_t) dB_t, \quad (8)$$

whose one-time marginals $p_t(x) dx = \text{Law}(X_t)$ satisfy $p_0(x) dx = p_{\text{simple}}(x) dx$ and $p_1(x) dx = p_{\text{data}}(x) dx$.

One can determine such a process through specifying tractable *conditional* paths, or stochastic interpolants, realized on one probability space. Let $(\Omega, \mathcal{F}, \mathbb{P})$ carry a d -dimensional Brownian motion $\{B_t\}_{0 \leq t \leq 1}$ (Definition 2.2.14) and a random variable $Z: (\Omega, \mathcal{F}) \rightarrow (\mathcal{Z}, \mathcal{B}(\mathcal{Z}))$ valued in a compact Polish space $\mathcal{Z}$, with law $p_Z := \text{Law}(Z)$, independent of $\sigma(B_s : 0 \leq s \leq 1)$. The latent is typically a pair of endpoints $Z = (X_0, X_1) \sim p_{\text{simple}} \otimes p_{\text{data}}$, and the compactness assumption is not particularly restrictive in practice, since datasets realized with floating point numbers on a computer are defined on a compact set. For each fixed realization z of Z , the conditional path solves

$$dX_t^z = f_t^z(X_t^z) dt + g_t^z(X_t^z) dB_t, \quad (9)$$

chosen so that its law, marginalized over z , has the desired endpoints. We only assume that the drift $f_t^z: \mathbb{R}^d \rightarrow \mathbb{R}^d$ and diffusion coefficient $g_t^z: \mathbb{R}^d \rightarrow \mathbb{R}^{d \times d}$ are measurable in (t, x) , and that the conditional process (9) is a diffusion with a well-defined generator $\mathcal{L}_t^z$.

Remark 3.1.1. We avoid trying to characterize all possible choices of f_t^z and g_t^z here. One could assume the linear growth and Lipschitz conditions in Theorem 2.2.21. But this would exclude Brownian bridges because the drift explodes as $t \uparrow 1$, even though these are well-behaved conditional paths on $[0, 1)$ and are used in Holderrieth et al. (2025). What matters for the generator-matching argument is that the resulting marginal process, or Markov mimic, satisfies our assumptions. We cannot verify this mathematically for the learned models considered here. The insight is largely experimental, namely that the generated samples look as if they come from p_{data} .

Next, we write $X_t^Z(\omega) := X_t^{Z(\omega)}(\omega)$, and for fixed t, z , we write

$$p_t^z(A) := \mathbb{P}(X_t^z \in A) = \text{Law}(X_t^Z \mid Z = z), \quad A \in \mathcal{B}(\mathbb{R}^d).$$

The marginal law is then

$$p_t(dx) := \text{Law}(X_t^Z) = \int p_t^z(dx) p_Z(dz).$$

Example 3.1.2. With $z = (x_0, x_1)$, the Brownian-bridge interpolant

$$dX_t^z = \frac{x_1 - X_t^z}{1-t} dt + \sigma dB_t$$

with the continuous extension $X_1^z = x_1$ has endpoint marginals $p_0^z = \delta_{x_0}$ and $p_1^z = \delta_{x_1}$ (and so is a stochastic interpolant, as defined above). Its conditional generator is singular as $t \uparrow 1$. This is acceptable for the conditional path construction, provided the resulting marginal process satisfies our assumptions (which we make no attempt to verify here). The $t = 1$ marginal is

$$p_1(dx) = \int \delta_{x_1}(dx) p_{\text{data}}(x_1) dx_1 = p_{\text{data}}(x) dx,$$

so $p_1 = p_{\text{data}}$, and likewise $p_0 = p_{\text{simple}}$.

The marginal coefficients are recovered from the conditional ones by conditioning on X_t^Z . Set $a_t^z(x) := g_t^z(x)g_t^z(x)^\top$. The generator for the conditional diffusion process is

$$\mathcal{L}_t^z f(x) = f_t^z(x) \cdot \nabla f(x) + \frac{1}{2} a_t^z(x) : \nabla^2 f(x), \quad f \in \mathcal{T},$$

where $A : B = \sum_{i,j} A_{ij} B_{ij}$ is the Frobenius inner product and $\nabla^2 f$ the Hessian. For each fixed z its law satisfies the Kolmogorov forward equation (Corollary 2.4.13)

$$\partial_t \mathbb{E}[f(X_t^z)] = \mathbb{E}[\mathcal{L}_t^z f(X_t^z)], \quad f \in \mathcal{T}.$$

Define the *marginal generator* by

$$\mathcal{L}_t f(x) := \mathbb{E}[\mathcal{L}_t^Z f(x) \mid X_t^Z = x].$$

As the diffusion generator is linear in its coefficients $(f_t^z(x), a_t^z(x))$, this conditional average has the form of a diffusion generator

$$\mathcal{L}_t f(x) = f_t(x) \cdot \nabla f(x) + \frac{1}{2} a_t(x) : \nabla^2 f(x),$$

with coefficients

$$f_t(x) = \mathbb{E}[f_t^Z(x) \mid X_t^Z = x], \quad a_t(x) = \mathbb{E}[a_t^Z(x) \mid X_t^Z = x]. \quad (10)$$

Averaging the conditional forward equation over $Z \sim p_Z$ and using the tower property, the law $p_t(dx)$ of X_t^Z satisfies

$$\partial_t \mathbb{E}[f(X_t^Z)] = \mathbb{E}[\mathcal{L}_t^Z f(X_t^Z)] = \mathbb{E}[\mathbb{E}[\mathcal{L}_t^Z f(X_t^Z) \mid X_t^Z]] = \mathbb{E}[\mathcal{L}_t f(X_t^Z)], \quad f \in \mathcal{T},$$

so $\{p_t\}$ solves the forward equation for $\mathcal{L}_t$. If $\mathcal{L}_t$ is the generator of a Markov process $\hat{X}_t$ on $\mathbb{R}^d$ and this equation determines $\{p_t\}$ uniquely (Assumption 2.4.14(v)), then $\hat{X}_t$ is a Markov mimic of $\{X_t^Z\}$ that discards Z and shares the marginals (p_t) . Thus, in the idealized population setting, the target coefficients (10) define a marginal Markov process that transports p_{simple} to p_{data} . The existence of a Feller process with generator $\mathcal{L}_t$ and the uniqueness of the KFE are assumed here, not proved from the conditional paths. The same conditioning idea is used below with an evolving latent process Z_t .

Therefore, under these assumptions, learning the marginal process reduces to the problem of learning a conditional average. A general class of loss functions which minimize at averages are Bregman divergences, and they enable us to turn the tractable interpolants into a neural-network training objective. In the idealized population setting, this objective recovers the marginal target. A trained neural network only approximates that target. We do not have a concrete or rigorous bound on how close the learned process is to the marginal Markov mimic, and we are not aware of such bounds in the literature for the models considered here.

Definition 3.1.3 (Bregman divergence). Let $(V, \langle \cdot, \cdot \rangle_V)$ be a finite-dimensional inner product space and $\phi: \mathcal{C} \subseteq V \rightarrow \mathbb{R}$ strictly convex and differentiable on a closed convex set $\mathcal{C}$. The associated *Bregman divergence* is

$$D_\phi(a, b) = \phi(a) - \phi(b) - \langle a - b, \nabla \phi(b) \rangle_V, \quad a, b \in \mathcal{C}.$$

Lemma 3.1.4 ((Billera et al., 2026a, Lemma B.1)). For an integrable $\mathcal{C}$ -valued random variable A with $\mathbb{E}[\|\phi(A)\|] < \infty$,

$$\operatorname{argmin}_{v \in \mathcal{C}} \mathbb{E}[D_\phi(A, v)] = \mathbb{E}[A].$$

Proof. One has that $\mathbb{E}[D_\phi(A, v)] - D_\phi(\mathbb{E}[A], v) = \mathbb{E}[D_\phi(A, \mathbb{E}[A])] \geq 0$. $\square$

Example 3.1.5. Taking $\phi(x) = \frac{1}{2}\|x\|^2$ gives $D_\phi(a, b) = \frac{1}{2}\|a - b\|^2$, the squared Euclidean loss.

Now, we write a subscript on $\mathbb{E}$ to list the laws of random variables in the following expectation, so that for instance $\mathbb{E}_{t \sim \text{Unif}(0,1), Z \sim p_Z, X_t^Z \sim p_t^Z}[\cdot]$ denotes the expectation under t uniformly on $[0, 1]$, then $Z \sim p_Z$, then $X_t^Z \sim p_t^Z$. The conditional regression loss

$$L(\theta) = \mathbb{E}_{t \sim \text{Unif}(0,1), Z \sim p_Z, X_t^Z \sim p_t^Z}[w(t) D_\phi(f_t^Z(X_t^Z), f_t^\theta(X_t^Z))] \quad (11)$$

is then minimized pointwise at the marginal coefficient $f_t(x) = \mathbb{E}[f_t^Z(x) \mid X_t^Z = x]$, assuming the relevant conditional expectations exist and the parametrization class contains this target. This can be seen by conditioning on X_t^Z ,

$$L(\theta) = \mathbb{E}_{t \sim \text{Unif}(0,1), X_t \sim p_t}[\mathbb{E}[w(t) D_\phi(f_t^Z(X_t), f_t^\theta(X_t)) \mid X_t^Z = X_t]].$$

Minimizing the outer expectation pointwise gives the conditional average above. An analogous loss learns the diffusion matrix a_t . In many applications, the diffusion matrix $a_t^z(x) := \sigma_t(x)$ is independent of z . In that case, the corresponding marginal diffusion coefficient $a_t(x) = \mathbb{E}[a_t^Z(x) \mid X_t^Z = x]$ coincides with the conditional one, and so the marginal diffusion coefficient is known *a priori*.

3.2 LATENT PROCESS GENERATOR MATCHING

The static-latent calculation above explains why regression against conditional targets gives a marginal process. Those identities were first intended for a fixed latent variable z . Branching Flows needs the same idea with a latent state Z_t that changes over time. The network observes X_t , while Z_t is used to form the training target and then marginalized out.

Rather than working with the generator only as an operator on test functions, one parametrizes with a so-called linear parametrization.

Definition 3.2.1 (Linear parametrization, (Holderrieth et al., 2025; Billera et al., 2026b)). For fixed t , a *linear parametrization* of a generator $\mathcal{L}_t$ acting on test functions $f \in \mathcal{T} \subset C_0(S)$ consists of, for each $x \in S$, an inner product space $(V_{t,x}, \langle \cdot, \cdot \rangle_{V_{t,x}})$, a closed convex subset $\mathcal{C} \subseteq V_{t,x}$, a linear map $\mathcal{K}_{t,x}: \mathcal{T} \rightarrow V_{t,x}$, and a measurable vector field $F_t(x) \in \mathcal{C}$, such that

$$\mathcal{L}_t f(x) = \langle \mathcal{K}_{t,x} f, F_t(x) \rangle_{V_{t,x}}, \quad f \in \mathcal{T}. \quad (12)$$

For conditional parametrizations $\mathcal{L}_t^{Z_t} f(x) = \langle \mathcal{K}_{t,x} f, F_t^{Z_t}(x) \rangle_{V_{t,x}}$ used below, we assume

$$\mathbb{E}[\|F_t^{Z_t}(X_t)\|_{V_{t,X_t}}] < \infty$$

and that, for the chosen family of Bregman divergences $\{D_{t,x}\}$, the corresponding losses have finite expectation (so that the loss function is well-defined).

579 **Example 3.2.2.** For a diffusion (Definition 2.5.5), take $V_{t,x} = \mathbb{R}^n \oplus \mathbb{R}^{n \times n}$ with inner product

$$\langle (a, A), (b, B) \rangle_{V_{t,x}} = a \cdot b + A : B,$$

580 where $A : B = \sum_{i,j} A_{ij} B_{ij}$ denotes the Frobenius inner product. With $\mathcal{K}_{t,x} f = (\nabla f(x), \frac{1}{2} \nabla^2 f(x))$ and
581 $F_t(x) = (b_t(x), a_t(x))$,

$$\langle \mathcal{K}_{t,x} f, F_t(x) \rangle_{V_{t,x}} = \nabla f(x) \cdot b_t(x) + \frac{1}{2} \nabla^2 f(x) : a_t(x).$$

582 Let $\mathcal{X}$ be a locally compact Polish space and let $\mathcal{Z}$ be a compact Polish space. Then $\mathcal{X} \times \mathcal{Z}$ is locally
583 compact Polish and

$$\mathcal{B}(\mathcal{X} \times \mathcal{Z}) = \mathcal{B}(\mathcal{X}) \otimes \mathcal{B}(\mathcal{Z}).$$

584 Let $(X_t, Z_t) : (\Omega, \mathcal{F}) \rightarrow (\mathcal{X} \times \mathcal{Z}, \mathcal{B}(\mathcal{X}) \otimes \mathcal{B}(\mathcal{Z}))$ be a Feller process on the product space, adapted to a
585 filtration $\{\mathcal{F}_t\}$, with generator $\mathcal{W}_t$ and law $p_t^{\mathcal{X} \times \mathcal{Z}} := \text{Law}(X_t, Z_t)$. Let $\pi_{\mathcal{X}} : \mathcal{X} \times \mathcal{Z} \rightarrow \mathcal{X}$ be the projection,
586 so $p_t^{\mathcal{X}} := (\pi_{\mathcal{X}})_\# p_t^{\mathcal{X} \times \mathcal{Z}} = \text{Law}(X_t)$. Since $\mathcal{Z}$ is compact, every $f \in C_0(\mathcal{X})$ gives $f \circ \pi_{\mathcal{X}} \in C_0(\mathcal{X} \times \mathcal{Z})$.
587 We assume that for every $f \in \mathcal{T}(\mathcal{X})$ this cylinder function lies in the joint test class/domain of $\mathcal{W}_t$. For
588 (x_t, z_t) and $f \in \mathcal{T}(\mathcal{X})$, define the *conditional generator*

$$\mathcal{L}_t^{z_t} f(x_t) := \mathcal{W}_t(f \circ \pi_{\mathcal{X}})(x_t, z_t), \quad (13)$$

589 which probes the joint dynamics through a test function that only sees the $\mathcal{X}$ -coordinate. Define the
590 *marginal generator* by

$$\mathcal{L}_t f(x_t) := \mathbb{E}[\mathcal{W}_t(f \circ \pi_{\mathcal{X}})(X_t, Z_t) \mid X_t = x_t]. \quad (14)$$

591 If $\mathcal{Z}$ is not compact, the statement remains possible but the C_0 -Feller formulation no longer automatically
592 admits $f \circ \pi_{\mathcal{X}}$, since this function need not vanish at infinity in the latent coordinate. In that case one
593 must either explicitly include these cylinder functions in the generator domain/test class, or pass to a
594 local martingale-problem formulation with localization on compact subsets.

595 **Theorem 3.2.3** (Marginal property of the projected process, (Billera et al., 2026a)). *Suppose (X_t, Z_t)*
596 *satisfies our assumptions (Assumption 2.4.14, lifted to $\mathcal{X} \times \mathcal{Z}$), that $f \circ \pi_{\mathcal{X}}$ lies in the joint test class for*
597 *every $f \in \mathcal{T}(\mathcal{X})$, and that the integrability condition $\mathbb{E}[|\mathcal{W}_t(f \circ \pi_{\mathcal{X}})(X_t, Z_t)|] < \infty$ holds for all $f \in \mathcal{T}(\mathcal{X})$.*
598 *Then the $\mathcal{X}$ -marginals $p_t^{\mathcal{X}} = (\pi_{\mathcal{X}})_\# p_t^{\mathcal{X} \times \mathcal{Z}}$ solve the weak KFE for $\mathcal{L}_t$:*

$$\partial_t \mathbb{E}[f(X_t)] = \mathbb{E}[\mathcal{L}_t f(X_t)], \quad f \in \mathcal{T}(\mathcal{X}).$$

599 *Proof.* Since the joint process satisfies our assumptions, its KFE says that, for every test function φ in
600 the joint test class,

$$\partial_t \mathbb{E}[\varphi(X_t, Z_t)] = \mathbb{E}[\mathcal{W}_t \varphi(X_t, Z_t)].$$

601 We apply this with $\varphi = f \circ \pi_{\mathcal{X}}$, which lies in the joint test class by assumption. Since $(f \circ \pi_{\mathcal{X}})(x, z) = f(x)$,
602 the left-hand side becomes

$$\partial_t \mathbb{E}[(f \circ \pi_{\mathcal{X}})(X_t, Z_t)] = \partial_t \mathbb{E}[f(X_t)].$$

603 For the right-hand side, the tower property and the definition of $\mathcal{L}_t$ give

$$\begin{aligned} \mathbb{E}[\mathcal{W}_t(f \circ \pi_{\mathcal{X}})(X_t, Z_t)] &= \mathbb{E}[\mathbb{E}[\mathcal{W}_t(f \circ \pi_{\mathcal{X}})(X_t, Z_t) \mid X_t]] \\ &= \mathbb{E}[\mathcal{L}_t f(X_t)]. \end{aligned}$$

604 Therefore

$$\partial_t \mathbb{E}[f(X_t)] = \mathbb{E}[\mathcal{L}_t f(X_t)],$$

605 which is the weak KFE for the marginal generator $\mathcal{L}_t$, against the marginal law $p_t^{\mathcal{X}}(dx)$. $\square$

606 **Remark 3.2.4.** The interpretation of the result

$$\partial_t \mathbb{E}[f(X_t)] = \mathbb{E}[\mathcal{L}_t f(X_t)],$$

607 is that $\mathcal{L}_t$ is the generator of a Markov process $\hat{X}_t$ with the same one-time marginals as X_t (i.e., a Markov
608 mimic of X_t , which may not alone be Markov), if the KFE for $\mathcal{L}_t$ is uniquely solved by these marginals as
609 assumed in Assumption 2.4.14(v). The regularity assumptions 2.4.14 are applied to $\hat{X}_t$, arising from the
610 marginal generator (which was specified by the interpolating conditional process (X_t, Z_t)). Therefore,
611 in the idealized population setting, a Markov process $\hat{X}_t$ generated by $\mathcal{L}_t$ initialized at the simple prior
612 distribution p_{simple} has the same one-time marginals as X_t (which again, may only be Markov as the pair
613 (X_t, Z_t)). In particular, when $p_0^{\mathcal{X}} = p_{\text{simple}}$ and $p_1^{\mathcal{X}} = p_{\text{data}}$, this Markov mimic transports the simple
614 prior distribution to the data distribution.

Remark 3.2.5. In Branching Flows we use a double conditioning structure. A static latent Z first samples the coupling between the prior and data samples. Conditional on this Z , an evolving “branch-index” process G_t plays the role of the latent process in the theorem. For this, one can regard (Z, G_t) as the latent process, with the static coordinate remaining constant in time.

If the conditional generator (13) admits a linear parametrization

$$\mathcal{L}_t^{z_t} f(x) = \langle \mathcal{K}_{t,x} f, F_t^{z_t}(x) \rangle_{V_{t,x}},$$

then by linearity, the marginal target is again a conditional average

$$F_t(x) = \mathbb{E}[F_t^{Z_t}(x) \mid X_t = x]. \quad (15)$$

Indeed, one has that

$$\begin{aligned} \mathcal{L}_t f(x) &= \mathbb{E}[\mathcal{L}_t^{Z_t} f(x) \mid X_t = x] \\ &= \mathbb{E}[\langle \mathcal{K}_{t,x} f, F_t^{Z_t}(x) \rangle_{V_{t,x}} \mid X_t = x] \\ &= \langle \mathcal{K}_{t,x} f, \mathbb{E}[F_t^{Z_t}(x) \mid X_t = x] \rangle_{V_{t,x}}, \end{aligned}$$

where the last equality uses the linearity of the inner product in the right slot (when, as above, the left slot is fixed). The integrability condition in Definition 3.2.1 is used here to ensure that the conditional expectation $\mathbb{E}[F_t^{Z_t}(x) \mid X_t = x]$ is well-defined. For a chosen family of Bregman divergences $\{D_{t,x}\}$, the same argument yields the gradient equality.

Theorem 3.2.6 (Gradient equality, (Billera et al., 2026a)). *Define*

$$\begin{aligned} L_{\text{cond}}(\theta) &= \mathbb{E}_{t, (X_t, Z_t) \sim p_t^{\mathcal{X} \times \mathcal{Z}}} [D_{t, X_t}(F_t^{Z_t}(X_t), F_t^\theta(X_t))], \\ L_{\text{marg}}(\theta) &= \mathbb{E}_{t, X_t \sim p_t^{\mathcal{X}}} [D_{t, X_t}(F_t(X_t), F_t^\theta(X_t))]. \end{aligned}$$

Under the integrability conditions above, assume also that the maps $\theta \mapsto D_{t,x}(F_t^z(x), F_t^\theta(x))$ are differentiable. We further assume the relevant derivatives are bounded by an integrable function, so that differentiation under the expectation is valid. Then $\nabla_\theta L_{\text{cond}}(\theta) = \nabla_\theta L_{\text{marg}}(\theta)$.

Thus one may sample (X_t, Z_t) , use Z_t to compute the target $F_t^{Z_t}(X_t)$, and train a network F_t^θ that receives only X_t .

The important modelling point is that the latent state Z_t is *not* passed to the neural network. It is used by the sampler to construct the training target, but the prediction itself is a function only of the visible state. The training procedure is stated in algorithmic notation in §4.

In applications the joint law $p_t^{\mathcal{X} \times \mathcal{Z}}$ is often constructed by first drawing a static latent z and then running (X_t, Z_t) conditionally on z . This keeps the original static-latent intuition while using the latent-process formulation needed here. One samples $z \sim p_Z$, samples (x_t, z_t) from the path conditioned on z , and trains $F_t^\theta(x_t)$ against the conditional target while withholding z_t from the network. Branching Flows uses this structure, with the latent process tracking branch membership, as developed in §5.

4 TRAINING BY STOCHASTIC GRADIENT DESCENT

The preceding section identifies the target that the network should learn. We now explain what it means to train the neural network that represents this target.

Training a neural network means approximately minimizing a loss function over its parameters θ . In the generator-matching notation used above, the network represents the visible target $F_t^\theta(x)$ in a linear parametrization of the generator. For a chosen family of Bregman divergences $\{D_{t,x}\}$, the ideal conditional objective has the form

$$L_{\text{cond}}(\theta) = \mathbb{E}[D_{t, X_t}(F_t^{Z_t}(X_t), F_t^\theta(X_t))],$$

with the Branching Flows specialization introduced later in §5. By Theorem 3.2.6, this conditional objective has the same gradient as the marginal objective, under the stated integrability and domination assumptions. The exact gradient is an expectation over times, conditional-path samples, and latent variables. In practice it is approximated by a finite batch, giving the stochastic-gradient update

$$\theta \leftarrow \theta - \eta \nabla_\theta \widehat{L}_{\text{cond}}(\theta), \quad \eta > 0, \quad (16)$$

where η is the *learning rate*. This is stochastic gradient descent (Robbins and Monro, 1951). In practice one typically uses an adaptive variant such as Adam (Kingma and Ba, 2015).

Algorithm 3. Training a generator-matching network.

Require: Sampler for triples (t, x, z) , conditional target rule, family of Bregman divergences $\{D_{t,x}\}$, batch size B , learning rate η .

1. Initialize the network parameters θ .

2. **repeat**

 Sample a batch $\{(t_j, x_j, z_j)\}_{j=1}^B$.

 For each j , set

$$y_j \leftarrow F_{t_j}^{z_j}(x_j), \quad \hat{y}_j \leftarrow F_{t_j}^{\theta}(x_j), \quad D_j \leftarrow D_{t_j, x_j}.$$

 Here z_j is used only to compute y_j . The network input is x_j .

 Compute

$$\hat{L}_{\text{cond}}(\theta) \leftarrow \frac{1}{B} \sum_{j=1}^B D_j(y_j, \hat{y}_j).$$

 Update $\theta \leftarrow \theta - \eta \nabla_{\theta} \hat{L}_{\text{cond}}(\theta)$, or apply the corresponding Adam update.

3. **until** satisfactory.

return θ

The sampler in Algorithm 3 is the conditional-path construction appropriate to the model. For stochastic interpolants it samples a time, a latent endpoint variable, and X_t . For Branching Flows (§5) it samples the static coupling, branch membership, and the corresponding split, deletion, and base targets. In all cases the latent variables are used to form the target, but are not passed as inputs to the neural network.

In Algorithm 3, iterating “until satisfactory” is deliberately ambiguous, as how long one should iterate for is driven by experimental results. In general though, one should *at least* iterate until the loss function has stopped meaningfully decreasing and appears to have converged. There have, for example, been interesting observations of neural networks improving substantially when trained for long periods after the loss function appeared to have already converged many iterations earlier, a phenomenon known as *grokking* (Power et al., 2022).

5 BRANCHING FLOWS

Branching Flows (Billera et al., 2026c) applies the Latent Process Generator Matching framework of §3.2 to *variable-length* generation, where the number of elements in a sample is not known in advance, as with the number of atoms in a molecule or residues in a protein. The state space is then not a fixed Euclidean space but the disjoint-union state space S_N of §2.6, and the generative process must be able to change the number of elements as it evolves. We give a simplified exposition with Euclidean base space $\mathcal{E} = \mathbb{R}^d$, although applications to products of discrete spaces, Euclidean spaces and Riemannian manifolds are discussed in Billera et al. (2026c).

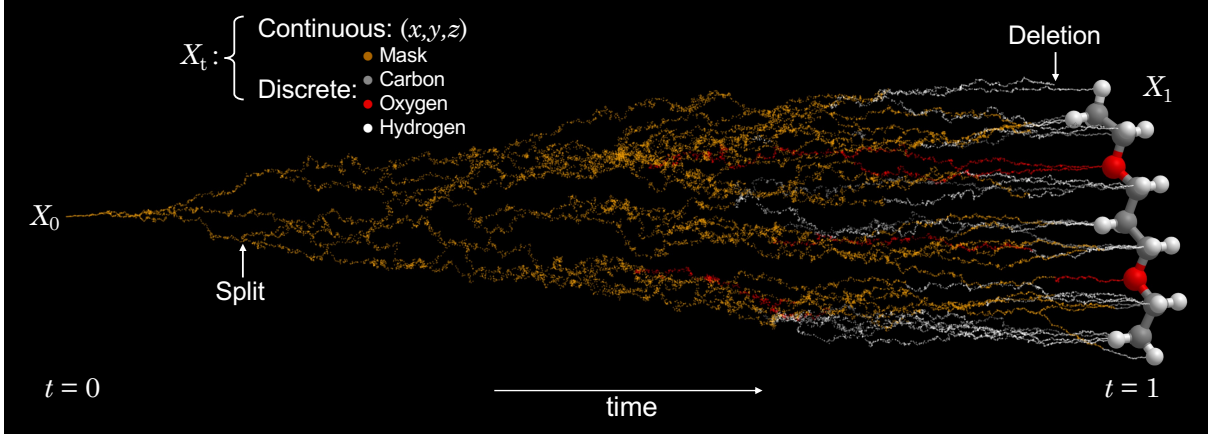


Figure 1: An illustrative branching-flow trajectory for molecule generation. Each atom carries a continuous component, its three-dimensional coordinate, and a discrete component (its atom type). The two components coevolve as the state X_t starts from a single atom of “masked”, or undetermined type, at X_0 towards a molecule-like sample X_1 , with atoms splitting and being deleted along the way so that the number of atoms is itself generated. Figure reproduced from (Billera et al., 2026c).

5.1 A LINEAR PARAMETRIZATION OF THE BRANCHING GENERATOR

For training we factor the per-element split and deletion rates of §2.6 into time-dependent *hazards* $h_{\text{split}}(t), h_{\text{del}}(t) > 0$ and per-element *multipliers*. For a state $(n, x) \in S_N$, we model the rate of splitting the i th element as $h_{\text{split}}(t) R_{t,i}(n, x)$ and the rate of deleting it as $h_{\text{del}}(t) \rho_{t,i}(n, x)$. Here $R_{t,i}(n, x) \geq 0$ is intended as the remaining number of splits ahead of element i and $\rho_{t,i}(n, x) \in [0, 1]$ is a deletion-intensity multiplier. Thus the infinitesimal probability of deleting element i in $(t, t + \delta]$ is $h_{\text{del}}(t) \rho_{t,i}(n, x) \delta + o(\delta)$, when the operation is admissible. At the boundaries of S_N , operations that would leave the state space are omitted, as in §2.6. With the atomic jump kernel of Definition 2.5.2,

$$Q_t(dy; (n, x)) = I_{\{n < N\}} \sum_{i=1}^n h_{\text{split}}(t) R_{t,i}(n, x) \delta_{\text{split}_i(n, x)}(dy) + I_{\{n > 1\}} \sum_{i=1}^n h_{\text{del}}(t) \rho_{t,i}(n, x) \delta_{\text{del}_i(n, x)}(dy),$$

the jump generator of §2.6 reads

$$\mathcal{L}_t^{\text{jump}} f(n, x) = I_{\{n < N\}} h_{\text{split}}(t) \sum_{i=1}^n (f(\text{split}_i(n, x)) - f(n, x)) R_{t,i}(n, x) + I_{\{n > 1\}} h_{\text{del}}(t) \sum_{i=1}^n (f(\text{del}_i(n, x)) - f(n, x)) \rho_{t,i}(n, x).$$

To cast this in the form of Definition 3.2.1 we equip $\mathbb{R}^n$ with the time-indexed family of inner products

$$\langle v, w \rangle_{t,n}^{\text{split}} := h_{\text{split}}(t) \langle v, w \rangle_{\mathbb{R}^n} \quad \text{and} \quad \langle v, w \rangle_{t,n}^{\text{del}} := h_{\text{del}}(t) \langle v, w \rangle_{\mathbb{R}^n},$$

which are genuine inner products since $h_{\text{split}}(t), h_{\text{del}}(t) > 0$, and we define the linear operators

$$\mathcal{K}_{t,n}^{\text{split}} f(n, x) = I_{\{n < N\}} (f(\text{split}_i(n, x)) - f(n, x))_{i=1}^n \in \mathbb{R}^n, \\ \mathcal{K}_{t,n}^{\text{del}} f(n, x) = I_{\{n > 1\}} (f(\text{del}_i(n, x)) - f(n, x))_{i=1}^n \in \mathbb{R}^n.$$

Collecting the multipliers into $R_t(n, x) = (R_{t,i}(n, x))_{i=1}^n \in \mathbb{R}_{\geq 0}^n$ and $\rho_t(n, x) = (\rho_{t,i}(n, x))_{i=1}^n \in [0, 1]^n$, the jump generator becomes

$$\mathcal{L}_t^{\text{jump}} f(n, x) = \langle \mathcal{K}_{t,n}^{\text{split}} f(n, x), R_t(n, x) \rangle_{t,n}^{\text{split}} + \langle \mathcal{K}_{t,n}^{\text{del}} f(n, x), \rho_t(n, x) \rangle_{t,n}^{\text{del}}.$$

Between jumps the component index n is fixed, and the base motion can e.g. be an ordinary diffusion on $\mathbb{R}^{dn}$. Writing $f_n(x) := f(n, x)$ for $x \in \mathbb{R}^{dn}$, with drift $b_{t,n}(n, x) \in \mathbb{R}^{dn}$ and diffusion matrix $a_{t,n}(n, x) \in \mathbb{R}^{dn \times dn}$, this has the form

$$\mathcal{L}_t^{\text{base}} f(n, x) = b_{t,n}(n, x) \cdot \nabla f_n(x) + \frac{1}{2} a_{t,n}(n, x) : \nabla^2 f_n(x).$$

We assume this base generator (diffusion, as above, or potentially including jump terms) admits its own linear parametrization

$$\mathcal{L}_t^{\text{base}} f(n, x) = \langle \mathcal{K}_{t,n}^{\text{base}} f(n, x), F_t^{\text{base}}(n, x) \rangle_{t,n,x}^{\text{base}}$$

over an inner product space $V_{t,n,x}^{\text{base}}$. Summing the jump and base parts, the branching generator has the linear parametrization

$$\begin{aligned} \mathcal{L}_t^{\text{branch}} f(n, x) = & \langle \mathcal{K}_{t,n}^{\text{split}} f(n, x), R_t(n, x) \rangle_{t,n}^{\text{split}} + \langle \mathcal{K}_{t,n}^{\text{del}} f(n, x), \rho_t(n, x) \rangle_{t,n}^{\text{del}} \\ & + \langle \mathcal{K}_{t,n}^{\text{base}} f(n, x), F_t^{\text{base}}(n, x) \rangle_{t,n,x}^{\text{base}}, \end{aligned}$$

with learnable target $F_t(n, x) = (R_t(n, x), \rho_t(n, x), F_t^{\text{base}}(n, x))$. Equivalently, this is one inner product on the direct sum of the split, deletion, and base inner product spaces. Indeed, with

$$\mathcal{K}_{t,n} f(n, x) := (\mathcal{K}_{t,n}^{\text{split}} f(n, x), \mathcal{K}_{t,n}^{\text{del}} f(n, x), \mathcal{K}_{t,n}^{\text{base}} f(n, x)),$$

and with the natural direct-sum inner product, one obtains

$$\mathcal{L}_t^{\text{branch}} f(n, x) = \langle \mathcal{K}_{t,n} f(n, x), F_t(n, x) \rangle_{t,n,x}.$$

5.2 CONDITIONAL PATHS AND THE BRANCH-INDEX LATENT PROCESS

Now we specify conditional paths as illustrated in Figure 2. As in the stochastic interpolants of §3.1, we *first* draw a *static* latent Z . Here Z couples the simple-prior sample to the data sample, and specifies a forest that assigns to each simple-prior element a binary tree whose leaves are the data elements it flows to. The internal nodes of the trees are the *anchors* for split events. When more branches arise than there are data elements to receive them, the surplus branches are marked for deletion. Along the path each current element steers toward the anchor of its branch. When a split event occurs, the element on that branch splits and its coordinate is duplicated. Since only knowing X_t does not determine which branch a current element belongs to, we use an auxiliary branch-index process G_t that records these branch labels during training, but is withheld from the model (to reduce architectural complexity and as a useful inductive bias for the model) as in §3.2. This is all done so that the conditional path terminates at the data sample, of a potentially different length from the starting sample from the simple prior distribution, almost surely.

The split and deletion times are specified through so-called hazard distributions (Billera et al., 2026c, Apps. E and I). We assume these distributions are absolutely continuous on $[0, 1)$ with continuous density with respect to Lebesgue measure. If H is such a distribution on $[0, 1]$, with density f_H , distribution function F_H , and survival function $S_H(t) = 1 - F_H(t)$, its hazard rate on $[0, 1)$ is

$$h(t) = \frac{f_H(t)}{S_H(t)}.$$

One can show (by interpreting the above as an ordinary differential equation) that

$$S_H(t) = \exp\left(-\int_0^t h(s) ds\right), \quad S_H(1) = 0 \iff \int_0^1 h(s) ds = \infty.$$

We choose these distributions so that $S_H(1) = 0$, so an active event occurs before the terminal time with probability one. If the i th element of $(n, x_1, \dots, x_n)$ on a given branch at time t has $R_{t,i}^{z,g}(x)$ splits ahead of it in this conditional path construction, where g tracks which branch each element is on, then the probability of seeing the next split in $(t, t + \Delta t]$ is

$$R_{t,i}^{z,g}(x) h_{\text{split}}(t) \Delta t + o(\Delta t).$$

Computationally, the actual recursive sampler (Billera et al., 2026c, App. I) samples the next waiting time conditional on the current time and the number of remaining events. One can show (Billera et al.,

2026c, Thm. C.1.3) that if $U \sim \text{Unif}(0, 1)$ and $R > 0$, then the next split-event time in this construction can be sampled from the conditional law

$$F_{H_{\text{split}}}^{-1}(1 - S_{H_{\text{split}}}(t)(1 - U)^{1/R}),$$

which enables the recursive sampling of split events up to time t and the efficient construction of conditional paths.

Deletion follows the same construction but with $\rho_{t,i}^{z,g}(x) \in \{0, 1\}$ determining whether a node on a given branch is to be deleted by $t = 1$. The same type of sampler can be used for the hazard rate $h_{\text{del}}(t)$.

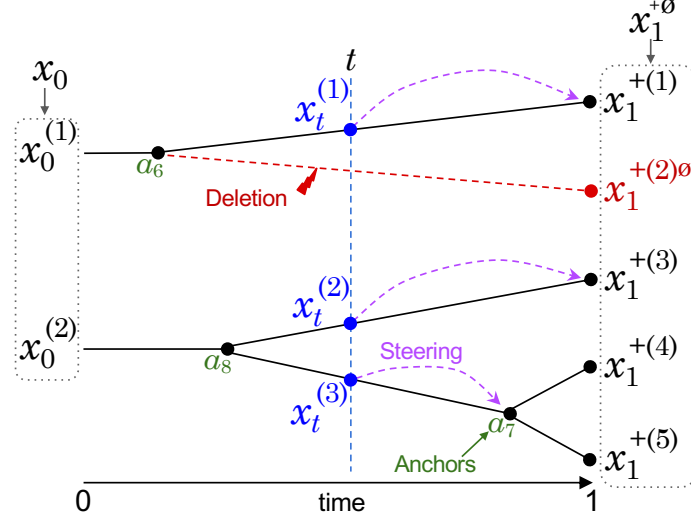


Figure 2: The conditional path from the Branching Flows paper itself (Billera et al., 2026c). The static latent Z couples the simple-prior elements $x_0^{(1)}, x_0^{(2)}$ to the data elements $x_1^{+(k)}$ through a forest of binary trees. Internal nodes have split-event times a_j . A surplus branch is assigned to deletion, drawn in red. Each current element $x_t^{(i)}$ steers toward the data endpoint of its branch, and the withheld index G_t records which branch each element sits on.

As a stochastic process, for training we apply LPGM to the pair $(X_t, (Z, G_t))$: the visible coordinate is X_t , and the latent coordinate is (Z, G_t) , with the static coordinate Z remaining constant in time. The split multiplier $R_{t,i}^{Z,G_t}$ is the remaining number of split events ahead of element i along its current branch subtree, at time t . Each time a split occurs along that branch, the remaining number of splits ahead of the node decreases by one. The deletion-intensity multiplier $\rho_{t,i}^{Z,G_t}$ records whether the branch has an active deletion clock. Along a conditional path it equals one exactly when the branch is destined for deletion (red in Figure 2), and is zero otherwise. In line with the lower-boundary convention of §2.6, these are constructed so as not to contain any deletions that would empty the sequence. In this way the conditional path reaches the data sample X_1 with the correct number of elements.

Algorithm 4 illustrates this conditional path construction, computationally.

Algorithm 4. Constructing a conditional branching path (Euclidean elements).

Require: data sample $x_1 = (x_1^{(1)}, \dots, x_1^{(m)}) \in (\mathbb{R}^d)^m$, simple prior distribution p_{simple} , split and deletion hazard distributions with their conditional waiting-time sampler (Billera et al., 2026c, App. I), bridge noise σ_t .

1. Sample a simple-prior state $x_0 = (x_0^{(1)}, \dots, x_0^{(n_0)}) \sim p_{\text{simple}}$.
2. Sample the coupling Z by assigning the m data elements among the n_0 simple-prior elements, building for each simple-prior element a binary tree whose leaves are its assigned data elements, and marking any surplus branches for deletion.
3. Assign split and deletion event times using the recursive conditional waiting-time construction from the hazard distributions (Billera et al., 2026c, App. I).
4. Sample a time $t \sim \text{Unif}(0, 1)$.
5. Form X_t and G_t by applying every split whose split-event time is at most t and every deletion whose deletion-event time is at most t , giving current elements $i = 1, \dots, n_t$ with branch labels G_t .
6. **for** each current element i on branch b , with root simple-prior element $x_0^{(b)}$:
 Set $R_{t,i}^{Z,G_t} \leftarrow$ the number of split-event times on b that exceed t .
 if b is destined for deletion:
 Draw $X_t^{(i)}$ from the deletion-branch coordinate law specified by the conditional path, set $\rho_{t,i}^{Z,G_t} \leftarrow 1$, and set $F_{t,i}^{Z,G_t,\text{base}}$ to the corresponding base quantity along that deletion branch.
 else
 Set $\rho_{t,i}^{Z,G_t} \leftarrow 0$, let $x_1^{(b)}$ be the data element assigned to b , and set $F_{t,i}^{Z,G_t,\text{base}} \leftarrow x_1^{(b)}$.
 Draw $X_t^{(i)} \leftarrow (1-t)x_0^{(b)} + tx_1^{(b)} + \sigma_t\sqrt{t(1-t)}\xi_i$ with $\xi_i \sim \mathcal{N}(0, I_d)$.
 end if
7. **end for**
return (X_t, G_t) and the conditional quantities R_t^{Z,G_t} , ρ_t^{Z,G_t} , and $F_t^{Z,G_t,\text{base}}$.

Remark 5.2.1. Algorithm 4 is a schematic description of the conditional-path sampler, for proofs and further details see Billera et al. (2026c). We do not attempt to characterize the regularity of these paths, or the resulting marginal process, in a mathematical way, but rather rely on successful experimental results.

Conditioning on (Z, G_t) makes the split multipliers $R_{t,i}^{Z,G_t}$, deletion-intensity multipliers $\rho_{t,i}^{Z,G_t}$, and base quantity $F_{t,i}^{Z,G_t,\text{base}}$ tractable, since given the branch assignment each element knows its own destination. For a state $X_t = (n_t, x)$, collect these conditional quantities into the direct-sum parameter

$$F_t^{Z,G_t,\text{branch}}(X_t) := (R_t^{Z,G_t}(X_t), \rho_t^{Z,G_t}(X_t), F_t^{Z,G_t,\text{base}}(X_t)).$$

The marginal branching parameter is the conditional average

$$F_t^{\text{branch}}(x) := \mathbb{E}[F_t^{Z,G_t,\text{branch}}(x) \mid X_t = x],$$

understood componentwise on each $\{n\} \times \mathbb{R}^{dn}$. This is the specialization of (15) to the branching direct-sum parametrization. A network F_t^θ receiving only X_t can be trained against $F_t^{Z,G_t,\text{branch}}(X_t)$. For Bregman divergences D^{split} and D^{del} on the split- and deletion-multiplier domains and taking the overall loss to be a sum of Bregman divergences on each component (which is again a Bregman divergence on the product space, cf. Billera et al. (2026b)), a valid conditional loss is

$$\begin{aligned} L_{\text{CBF}}(\theta) = \mathbb{E}_{\substack{t \sim \text{Unif}(0,1), Z \sim p_Z \\ (X_t, G_t) \sim p_t(dx, dg|Z)}} & \left[\sum_{i=1}^{n_t} D_{t,X_t,i}^{\text{split}}(R_{t,i}^{Z,G_t}(X_t), R_{t,i}^\theta(X_t)) \right. \\ & + \sum_{i=1}^{n_t} D_{t,X_t,i}^{\text{del}}(\rho_{t,i}^{Z,G_t}(X_t), \rho_{t,i}^\theta(X_t)) \\ & \left. + \sum_{i=1}^{n_t} D_{t,X_t,i}^{\text{base}}(F_{t,i}^{Z,G_t,\text{base}}(X_t), F_{t,i}^{\theta,\text{base}}(X_t)) \right]. \end{aligned} \quad (17)$$

We assume the model parametrization enforces $R_{t,i}^\theta(X_t)$, $\rho_{t,i}^\theta(X_t)$, and $F_{t,i}^{\theta,\text{base}}$ to lie in the domains of these divergences. In the idealized population setting, minimizing this objective recovers F_t^{branch}

parametrizing a marginal generator $\mathcal{L}_t^{\text{branch}}$ which transports samples from p_{simple} to p_{data} of potentially varying length, assuming all processes involved and the corresponding KFE are well-posed. We do not attempt to characterize when this is the case. The trained neural network only approximates these quantities, and we do not have a rigorous bound on the distance between the learned process and the ideal marginal Markov mimic. In any case, Branching Flows has obtained state-of-the-art performance, empirically, when fine-tuned into existing models for protein design to provide variable-length sampling.

Remark 5.2.2. The Branching Flows paper uses loss functions involving logarithms for the split and deletion multipliers. These require a slightly more general domain than the Bregman divergences defined here. See Billera et al. (2026c,b) for a discussion.

5.3 GENERATION IN THE EUCLIDEAN CASE

Once the multipliers and the base quantity are trained, sampling is treated as simulating the learned approximation to the idealized marginal branching generator $\mathcal{L}_t^{\text{branch}}$ on S_N with $\mathcal{E} = \mathbb{R}^d$. This is the Euler sampling of Algorithm 2 of §2.6 run with the learned rates $\lambda_{t,i}^\theta(x) = h_{\text{split}}(t) R_{t,i}^\theta(x)$ and $\mu_{t,i}^\theta(x) = h_{\text{del}}(t) \rho_{t,i}^\theta(x)$. The base drift is recovered for instance from an endpoint prediction x_1^θ through $u_t^\theta(x) = (x_1^\theta(x) - x)/(1 - t)$. The branch-index process G_t is never simulated and is immaterial during inference, due to the flexibility afforded by Latent Process Generator Matching in Theorem 3.2.3.

6 FEYNMAN-KAC STEERING AND ITS APPLICATION TO BRANCHING FLOWS

Having trained a model that samples from $p_1 \approx p_{\text{data}}$, one often wants to sample not from p_{data} itself but from a *reward-tilted* version of it. Given a reward $r: S_N \rightarrow \mathbb{R}$ scoring desirable attributes, such as the binding affinity of a protein binder, the target may e.g. become

$$p_1^{\text{tilt}}(\mathrm{d}x) = \frac{1}{Z_{\text{FK}}} e^{\beta r(x)} p_1(\mathrm{d}x), \quad Z_{\text{FK}} = \int e^{\beta r(x)} p_1(\mathrm{d}x),$$

or, more generally, $p_1^{\text{tilt}}(\mathrm{d}x) = w(x) p_1(\mathrm{d}x) / \mathbb{E}[w(X_1)]$ for a nonnegative weight w . Many techniques for this rely on the score $\nabla \log p_t$ or on a tractable likelihood. Feynman-Kac steering (Singhal et al., 2025) instead reweights the simulated trajectories of the generative process and requires neither, only the ability to simulate the process and to evaluate the reward. We describe the variant that reweights by the terminal reward $w(X_1)$.

6.1 TERMINAL REWEIGHTING AS A FEYNMAN-KAC PATH MEASURE

To sample on a computer we discretize the generative process on a grid $0 = t_1 < \dots < t_K = 1$, producing transition kernels $M_k(x_k, \mathrm{d}x_{k+1})$, for example an Euler-Maruyama step (§2.3) together with the branching updates of §5.3. The law of the discretized trajectory $x_{1:K}$ is

$$\mathbb{P}(\mathrm{d}x_{1:K}) = \mu(\mathrm{d}x_1) \prod_{k=1}^{K-1} M_k(x_k, \mathrm{d}x_{k+1}),$$

with μ the law of the initial state. Reweighting the terminal marginal by w corresponds to the tilted path law

$$\mathbb{Q}(\mathrm{d}x_{1:K}) = \frac{w(x_K)}{\mathbb{E}[w(x_K)]} \mathbb{P}(\mathrm{d}x_{1:K}),$$

whose terminal marginal is the desired $\mathbb{Q}(\mathrm{d}x_K) = w(x_K) \mathbb{P}(\mathrm{d}x_K) / \mathbb{E}[w(x_K)]$. More generally one writes the tilt as a product of nonnegative *potentials* Ψ_k . The original Feynman-Kac paper uses the notation G_k for these potentials. We use Ψ_k because G_t already denotes the branch-index process in Branching Flows.

$$\mathbb{Q}(\mathrm{d}x_{1:K}) = \frac{1}{Z_{\text{FK}}} \Psi_1(x_1) \mu(\mathrm{d}x_1) \prod_{k=1}^{K-1} M_k(x_k, \mathrm{d}x_{k+1}) \Psi_{k+1}(x_k, x_{k+1}),$$

subject to the single constraint

$$\Psi_1(x_1) \prod_{k=1}^{K-1} \Psi_{k+1}(x_k, x_{k+1}) = w(x_K), \quad Z_{\text{FK}} = \mathbb{E}[w(x_K)],$$

where Z_{FK} is unknown in practice. Two natural choices are *terminal-only* weighting, $\Psi_k \equiv 1$ for $k < K$ and $\Psi_K = w$, and *incremental* weighting, $\Psi_{k+1} = H_{k+1}/H_k$ for positive functions H_k with $H_K = w$, which telescopes to $w(x_K)$ and can reduce the variance of the procedure below.

6.2 SAMPLING BY SEQUENTIAL MONTE CARLO

The constant $Z_{\text{FK}} = \mathbb{E}[w(x_K)]$ is intractable, so $\mathbb{Q}$ cannot be sampled directly. Sequential Monte Carlo methods (Naesseth et al., 2019) sample it one step at a time by approximating the normalized tilted transition kernel

$$\widetilde{M}_k(x_k, dx_{k+1}) = \frac{\Psi_{k+1}(x_k, x_{k+1}) M_k(x_k, dx_{k+1})}{\mathbb{E}[\Psi_{k+1}(x_k, x_{k+1}) \mid x_k]}$$

with a population of N weighted particles. At step k one holds particles $\{x_k^n\}$ with weights $w_k^n = \Psi_k(\cdot)$ obtained from the previous propagation. After normalization $\bar{w}_k^n = w_k^n / \sum_m w_k^m$, the weighted empirical measure $\sum_n \bar{w}_k^n \delta_{x_k^n}$ converges, as $N \rightarrow \infty$, to the tilted one-step law by the law of large numbers, since for bounded φ

$$\int \varphi d(\sum_n \bar{w}_k^n \delta_{x_k^n}) = \frac{\frac{1}{N} \sum_n w_k^n \varphi(x_k^n)}{\frac{1}{N} \sum_n w_k^n} \xrightarrow{N \rightarrow \infty} \frac{\mathbb{E}[w_k \varphi]}{\mathbb{E}[w_k]}.$$

One then resamples ancestor indices, propagates each survivor through M_k , and reweights by Ψ_{k+1} , as in Algorithm 5. Resampling concentrates the population on high-reward regions of trajectory space without ever evaluating Z_{FK} .

Algorithm 5. Feynman–Kac steering by Sequential Monte Carlo.

Require: kernels M_k , potentials Ψ_k , initial law μ , particle count N , steps K .

1. Sample $x_1^n \sim \mu$ for $n = 1, \dots, N$ and set $w_1^n \leftarrow \Psi_1(x_1^n)$.
 2. **for** $k = 1, \dots, K - 1$:
 - Normalize $\bar{w}_k^n \leftarrow w_k^n / \sum_m w_k^m$.
 - Resample ancestors $A_k^n \sim \text{Categorical}(\bar{w}_k^{1:N})$ for $n = 1, \dots, N$.
 - Propagate $x_{k+1}^n \sim M_k(x_k^{A_k^n}, dx)$.
 - Reweight $w_{k+1}^n \leftarrow \Psi_{k+1}(x_k^{A_k^n}, x_{k+1}^n)$.
 3. **end for**
 4. Normalize $\bar{w}_K^n \leftarrow w_K^n / \sum_m w_K^m$.
- return** weighted terminal particles $\{(x_K^n, \bar{w}_K^n)\}_{n=1}^N$ approximating $\mathbb{Q}(dx_K) \propto w(x_K) \mathbb{P}(dx_K)$.

6.3 APPLICABILITY TO BRANCHING FLOWS

Feynman–Kac steering is well suited to Branching Flows precisely because it is *likelihood-free*.

- *It needs a simulator, not a likelihood.* The recursion uses only the one-step kernel M_k , which for Branching Flows is exactly the base, split, and deletion update of Algorithm 2, together with pointwise evaluations of the reward w . The branching state space S_N is trans-dimensional and the model exposes (to our knowledge) no tractable likelihood or score $\nabla \log p_t$ over it, so guidance and score-based steering do not apply, whereas Sequential Monte Carlo steering does.
- *Variable length is no obstacle.* Resampling acts on entire particles, that is, on complete variable-length states $(n, x) \in S_N$, regardless of their dimension. The only requirement on the potentials is that they be evaluable on a state, nothing is differentiated, and no dimensions need to be matched across particles.
- *The reward may be terminal and non-differentiable.* Terminal-only potentials $\Psi_K = w$ require nothing but the final reward $w(X_1)$, for example a physically computed energy of a generated structure, which need not be differentiable. Cheap intermediate proxies may be inserted as Ψ_k to reduce variance.

These are the same practical advantages exploited in Proteina Complexa (Didi et al., 2026), a state-of-the-art protein-design method, and the simulator-and-reward viewpoint carries over naturally to Branching Flows.

6.4 EXPERIMENTS ON QM9

We test Feynman–Kac-style steering on a pretrained QM9 branching-flow generator. QM9 is a standard quantum-chemistry benchmark consisting of roughly 134,000 small organic molecules, with equilibrium geometries and molecular properties computed by density functional theory (Ramakrishnan et al., 2014).

A molecular state carries a Euclidean component for the three-dimensional atom coordinates together with a discrete component for the atom identities, and these two coevolve along the branching flow as atoms are born, move, and die, so that the number of atoms is itself generated rather than fixed in advance. We use the model and its parallel sampler without retraining and refer the reader to (Billera et al., 2026c) for the architecture and training of the QM9 model and further details on discrete diffusion. To illustrate the method, we fix *one target molecule in advance* by drawing it once from the QM9 reference set. For a generated molecule x we form its RDKit fingerprint, a binary vector encoding the presence of local molecular substructures (Landrum, 2006). We score it by the Tanimoto distance (Tanimoto, 1958) $d(x, x_*)$ to the target fingerprint. We then use the terminal reward $R_\lambda(x) = \epsilon + \exp(-\lambda d(x, x_*))$ with a small floor $\epsilon = 10^{-9}$. The intensity λ sharpens the preference for molecules close to the target, and we compare a moderate setting $\lambda = 8$ with a sharper setting $\lambda = 16$. Summarizing, the reward function here is a measure of how close the generated molecule is to a fixed target molecule (which is not a practically useful task, but merely for illustration/experimentation).

The implementation uses per-molecule resampling at a few late checkpoints of the 1000 step trajectory. Each candidate is rolled out to a complete molecule for scoring, but at non-final checkpoints only the checkpoint state is kept. This should be read as a practical lookahead-resampling approximation, of the kind used in Proteina Complexa (Didi et al., 2026), rather than an exact estimator for a fully specified Feynman–Kac path law.

One way to write the idealized path law behind this approximation is as follows. Let $c_1 < \dots < c_J = K$ be the resampling checkpoints, and let

$$H_{c_j}(x_{c_j}) := \mathbb{E}[R_\lambda(\tilde{X}_K) \mid \tilde{X}_{c_j} = x_{c_j}]$$

denote the expected terminal reward of an independent rollout from checkpoint state x_{c_j} to time K . Then the corresponding checkpoint-tilted path law would be

$$\mathbb{Q}_{\text{look}}(\mathrm{d}x_{1:K}) = \frac{1}{Z_{\text{look}}} \left(\prod_{j=1}^J H_{c_j}(x_{c_j}) \right) \mathbb{P}(\mathrm{d}x_{1:K}).$$

In the experiment, H_{c_j} is replaced by a one-rollout Monte Carlo estimate $R_\lambda(\tilde{X}_K)$, so this is only a noisy practical approximation. Because the reward is applied at several late checkpoints, the product of lookahead potentials is generally steeper than a single terminal reward. Empirically, this extra selection pressure counteracts some discretization error in the sampler.

The analytically reweighted reference distribution is obtained directly from the finite QM9 reference set. If $y_1, \dots, y_M$ are the reference molecules, we compute the same terminal rewards $R_\lambda(y_m)$ and form the weighted empirical measure

$$\hat{q}_\lambda(\mathrm{d}y) := \sum_{m=1}^M \frac{R_\lambda(y_m)}{\sum_{\ell=1}^M R_\lambda(y_\ell)} \delta_{y_m}(\mathrm{d}y).$$

If the reference molecules are regarded as i.i.d. samples from the QM9 data law p_{QM9} , then for any bounded test function φ , the law of large numbers gives

$$\int \varphi(y) \hat{q}_\lambda(\mathrm{d}y) \xrightarrow{M \rightarrow \infty} \frac{\mathbb{E}_{Y \sim p_{\text{QM9}}}[R_\lambda(Y) \varphi(Y)]}{\mathbb{E}_{Y \sim p_{\text{QM9}}}[R_\lambda(Y)]},$$

which is exactly the terminal Feynman–Kac reward tilt of the reference data law. Figure 3 compares FK-style lookahead resampling with this analytically reweighted reference distribution for the same terminal reward. The samples are embedded in a shared UMAP projection of fingerprint space. UMAP, short for Uniform Manifold Approximation and Projection, is a nonlinear dimension-reduction method used here only to visualize high-dimensional RDKit fingerprints in two dimensions (McInnes et al., 2018). For both $\lambda = 8$ and $\lambda = 16$, the FK samples move toward the same target region as the analytically reweighted reference. The concentration is stronger for $\lambda = 16$, as expected from the sharper reward. This comparison should simply be read as qualitative evidence that the lookahead-resampling approximation captures the intended reward tilt.

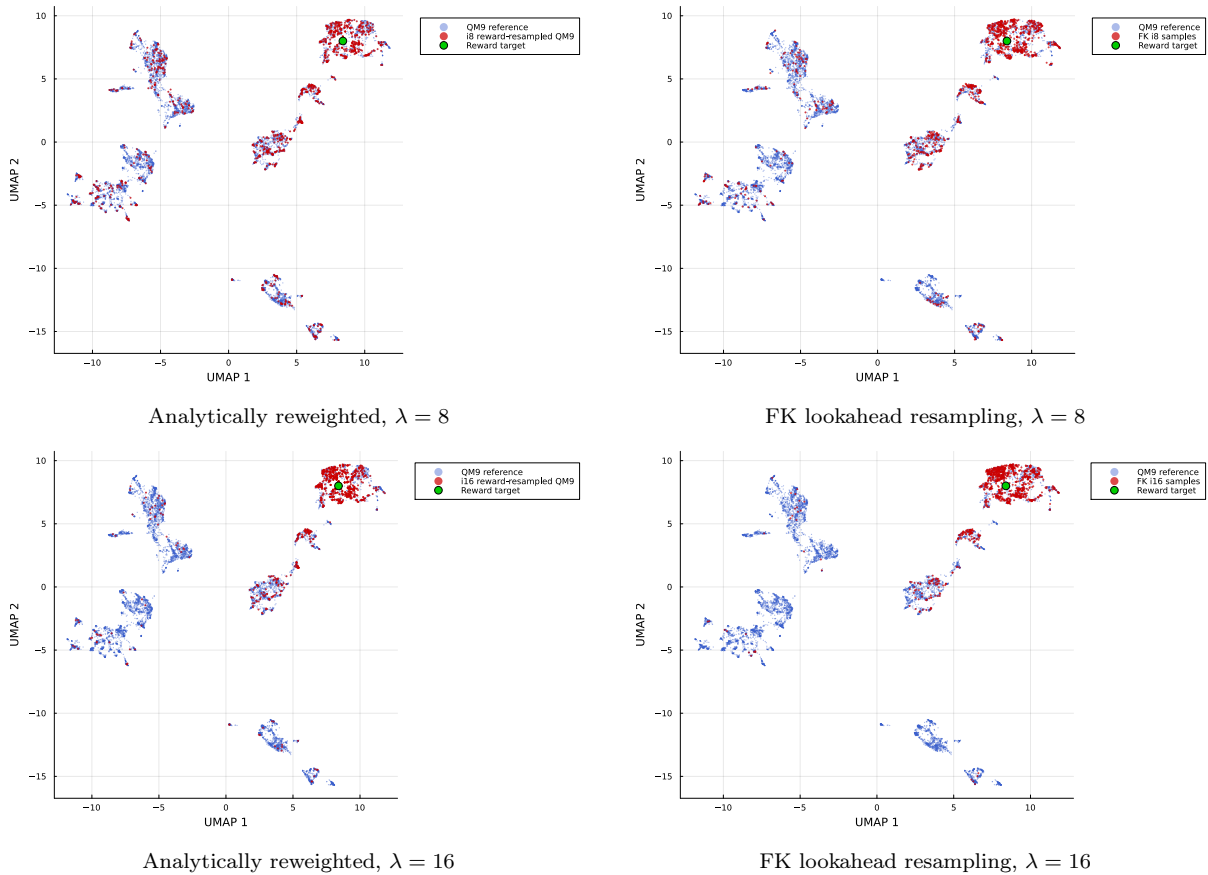


Figure 3: Shared UMAP projection of fingerprint space comparing the analytically reweighted reference distribution with FK-style lookahead-resampling samples. The two rows correspond to reward intensities $\lambda = 8$ and $\lambda = 16$. The FK samples concentrate in the same region as the analytically reweighted reference, with stronger concentration for the sharper reward.

REFERENCES

- Michael S Albergo, Nicholas M Boffi, and Eric Vanden-Eijnden. Stochastic interpolants: A unifying framework for flows and diffusions. *arXiv e-prints*, pages arXiv-2303, 2023.
- Brian D. O. Anderson. Reverse-time diffusion equation models. *Stochastic Processes and their Applications*, 12(3):313–326, 1982. doi: 10.1016/0304-4149(82)90051-5.
- Helen M Berman, John Westbrook, Zukang Feng, Gary Gilliland, T N Bhat, Helge Weissig, Ilya N Shindyalov, and Philip E Bourne. The Protein Data Bank. *Nucleic Acids Research*, 28(1):235–242, 2000.
- Lukas Billera, Hedwig Nora Nordlinder, and Ben Murrell. Latent process generator matching. In *ICLR 2026 2nd Workshop on Deep Generative Model in Machine Learning: Theory, Principle and Efficacy*, 2026a. URL <https://openreview.net/forum?id=XhmeZjNOAg>.
- Lukas Billera, Hedwig Nora Nordlinder, and Ben Murrell. Time dependent loss reweighting for flow matching and diffusion models is theoretically justified. In *ICLR 2026 2nd Workshop on Deep Generative Model in Machine Learning: Theory, Principle and Efficacy*, 2026b. URL <https://openreview.net/forum?id=COJIuGOeRN>.
- Lukas Billera, Hedwig Nora Nordlinder, Jack Collier Ryder, Anton Oresten, Aron Stålmarch, Theodor Mosetti Björk, and Ben Murrell. Branching flows: Discrete, continuous, and manifold flow matching with splits and deletions, 2026c. URL <https://arxiv.org/abs/2511.09465>.
- Vladimir I. Bogachev. *Measure Theory*. Measure Theory ; 1. Springer Berlin Heidelberg, Berlin, Heidelberg, 1st ed. 2007. edition, 2007. ISBN 1-280-74570-3.

- Philippe Courrège. Sur la forme intégrô-différentielle des opérateurs de c_k^∞ dans c satisfaisant au principe du maximum. *Séminaire Brelot-Choquet-Deny. Théorie du Potentiel*, 10(1):1–38, 1965–1966. URL https://www.numdam.org/item/SBCD_1965-1966__10_1_A2_0/. Talk no. 2.
- Valentin De Bortoli, Emile Mathieu, Michael Hutchinson, James Thornton, Yee Whye Teh, and Arnaud Doucet. Riemannian score-based generative modelling. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL <https://arxiv.org/abs/2202.02763>.
- Kieran Didi, Zuobai Zhang, Guoqing Zhou, Danny Reidenbach, Zhonglin Cao, Sooyoung Cha, Tomas Geffner, Christian Dallago, Jian Tang, Michael M. Bronstein, Martin Steinegger, Emine Kucukbenli, Arash Vahdat, and Karsten Kreis. Scaling atomistic protein binder design with generative pretraining and test-time compute, 2026. URL <https://arxiv.org/abs/2603.27950>.
- Albert Einstein. Über die von der molekularkinetischen Theorie der Wärme geforderte Bewegung von in ruhenden Flüssigkeiten suspendierten Teilchen. *Annalen der Physik*, 322(8):549–560, 1905. doi: 10.1002/andp.19053220806.
- Stewart N. Ethier and Thomas G. Kurtz. *Markov Processes: Characterization and Convergence*. Wiley Series in Probability and Mathematical Statistics. John Wiley & Sons, New York, 1986. doi: 10.1002/9780470316658.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models, 2020. URL <https://arxiv.org/abs/2006.11239>.
- Peter Holderrieth, Marton Havasi, Jason Yim, Neta Shaul, Itai Gat, Tommi Jaakkola, Brian Karrer, Ricky T. Q. Chen, and Yaron Lipman. Generator matching: Generative modeling with arbitrary Markov processes, 2025. URL <https://arxiv.org/abs/2410.20587>.
- Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the design space of diffusion-based generative models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL <https://arxiv.org/abs/2206.00364>.
- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015. URL <https://arxiv.org/abs/1412.6980>.
- Peter E. Kloeden and Eckhard Platen. *Numerical Solution of Stochastic Differential Equations*, volume 23 of *Applications of Mathematics*. Springer, Berlin, 1992. doi: 10.1007/978-3-662-12616-5.
- Timo Koski. Lecture notes: Probability and random processes at KTH (sf2940 probability theory). Lecture notes, KTH Royal Institute of Technology, Stockholm, 2017.
- Greg Landrum. RDKit: Open-source cheminformatics, 2006. URL <https://www.rdkit.org/>.
- Paul Langevin. Sur la théorie du mouvement brownien. *Comptes Rendus de l’Académie des Sciences (Paris)*, 146:530–533, 1908.
- Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling, 2023. URL <https://arxiv.org/abs/2210.02747>.
- Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete diffusion modeling by estimating the ratios of the data distribution. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.
- Leland McInnes, John Healy, and James Melville. UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv e-prints*, page arXiv:1802.03426, 2018. URL <https://arxiv.org/abs/1802.03426>.
- Christian A. Naesseth, Fredrik Lindsten, and Thomas B. Schön. Elements of sequential monte carlo. *Foundations and Trends® in Machine Learning*, 12(3):307–392, 2019.
- Bernt Øksendal. *Stochastic Differential Equations: An Introduction with Applications*. Universitext. Springer, Berlin, Heidelberg, 5 edition, 1998. doi: 10.1007/978-3-662-03620-4.

- 927 Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. Grokking: Gen-
 928 eralization beyond overfitting on small algorithmic datasets. *CoRR*, abs/2201.02177, 2022. URL
 929 <http://arxiv.org/abs/2201.02177>.
- 930 Raghunathan Ramakrishnan, Pavlo O. Dral, Matthias Rupp, and O. Anatole von Lilienfeld. Quantum
 931 chemistry structures and properties of 134 kilo molecules. *Scientific Data*, 1:140022, 2014. doi: 10.
 932 1038/sdata.2014.22.
- 933 Herbert Robbins and Sutton Monro. A stochastic approximation method. *The Annals of Mathematical*
 934 *Statistics*, 22(3):400–407, 1951. doi: 10.1214/aoms/1177729586.
- 935 Walter Rudin. *Real and Complex Analysis*. McGraw-Hill, New York, 3 edition, 1987.
- 936 Raghav Singhal, Zachary Horvitz, Ryan Teehan, Mengye Ren, Zhou Yu, Kathleen McKeown, and Rajesh
 937 Ranganath. A general framework for inference-time scaling and steering of diffusion models, 2025. URL
 938 <https://arxiv.org/abs/2501.06848>.
- 939 Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole.
 940 Score-based generative modeling through stochastic differential equations. In *International Conference*
 941 *on Learning Representations (ICLR)*, 2021. URL <https://arxiv.org/abs/2011.13456>.
- 942 Daniel W. Stroock and S. R. Srinivasa Varadhan. *Multidimensional Diffusion Processes*, volume 233 of
 943 *Grundlehren der mathematischen Wissenschaften*. Springer, Berlin, 1979.
- 944 Denis Talay and Luciano Tubaro. Expansion of the global error for numerical schemes solving
 945 stochastic differential equations. *Stochastic Analysis and Applications*, 8(4):483–509, 1990. doi:
 946 10.1080/07362999008809220.
- 947 Taffee T. Tanimoto. An elementary mathematical theory of classification and prediction. Technical
 948 report, International Business Machines Corporation, 1958.
- 949 Norbert Wiener. Differential-space. *Journal of Mathematics and Physics*, 2(1–4):131–174, 1923. doi:
 950 10.1002/sapm192321131.